

高三的时候，大概是四五月份吧，有篇作文写的是“我的理想”。当时觉得这么直球的命题绝对是拿来让大家给自己写鸡汤的，老师肯定都会给高分，所以也就放开了写。我的题目是“在无穷纸带之上”，您看，这 TOC 不就伏笔了吗？

这个领域确实很美，图灵机有种蒸汽朋克的机械感，同时保持着数学的精密感，人们在这台复杂程度恰到好处的机器上创立了无数漂亮的理论，虽然.....Open problem 仍然遍地都是。我大概率不会再深入这个领域了，这门课程算是对“在无穷纸带之上”的第一次也是最后一次真正的 call back。即使是走马观花，也给了我很棒的经验！

嗯.....课程内容特别难，作业题也真的真的特别难，我第五次作业每道题差不多要磨三个多小时（但成就感 up up! ），第六次作业更是向 GPT 投降，但考试题目的风格确实不会那么变态。这门课的期末会带给你巨大的焦虑，这很正常，我在这里说什么都无法消除你的焦虑，但你要知道，这很正常，这不是你的问题。

考前多看往年题，不用更直白了吧？

## 第一章 自动机与语言

§ 1.1 DFA 与 NFA

§ 1.2 正则表达式

§ 1.3 正则性判定

§ 1.4 CFL 与 PDA

## 第二章 图灵机

§ 2.1 确定性图灵机

§ 2.2 可计算性

## 第三章 图灵机时间复杂性

§ 3.1 时间复杂性

§ 3.2 时间完全性问题

§ 3.3 NP 相关的复杂性类

## 第四章 图灵机空间复杂性

§ 4.1 空间复杂性

§ 4.2 空间完全性问题

§ 4.3 小空间算法: NL 与 coNL

§ 4.4 多项式层级

## 第五章 布尔电路

§ 5.1 基本定义

§ 5.2 电路复杂性

§ 5.3 浅层电路

## 第六章 概率图灵机

§ 6.1 PTM 的定义与基本性质

§ 6.2 概率复杂性类小结

## 第七章 交互式证明

§ 7.1 IP 的定义与基本性质

§ 7.2 公共随机与 AM

§ 7.3 IP 与 PSPACE

## 第八章 量子计算理论初步\*

§ 8.1 量子态与操作

§ 8.2 量子计算模型与 BQP

## 第九章 伪随机理论\*

§ 9.1 密码学中的伪随机

§ 9.2 空间有界计算的伪随机

## 附录: TOC/advice

知识点

经典证明

数学工具

# 第一章 自动机与语言

## § 1.1 DFA 与 NFA

### 定义 1.1.1 (DFA)

称资料  $M = (Q, \Sigma, \delta, q_0, F)$  是确定性有限状态自动机 (DFA), 当

- $Q$  是有限状态集;
- $\Sigma$  是有限字符集;
- $\delta: Q \times \Sigma \rightarrow Q$  是转移函数;
- $q_0 \in Q$  是起始状态;
- $F \subset Q$  是接收状态集.

若字符串  $w_1, \dots, w_n = w \in \Sigma^*$  在输入  $M$  后到达接受状态  $q \in F$ , 即

$$\exists(r_0, \dots, r_n), r_0 = q_0 \wedge (\forall i \in [1 : n], \delta(r_{i-1}, w_i) = r_i) \wedge r_n \in F,$$

则称  $w$  被  $M$  接受.

### 定义 1.1.2 (语言/正则语言)

$A := \{w \in \Sigma^* : w \text{ accepted by } M\}$  称为由  $M$  定义的语言. 若语言  $A \subset \Sigma^*$  能被某种 DFA 识别, 则称它是一个正则语言.

我们知道  $|\mathcal{P}(\Sigma^*)| = \aleph_1$ , 也容易说明  $|\{M : \text{automanta with given } \Sigma\}| = \aleph_0$ , 所以一定存在非正则的语言.

这样的非正则的语言其实也容易举出例子:  $\Sigma = \{0, 1\}$ ,  $A = \{w : \#\{0 \in w\} = \#\{1 \in w\}\}$ . Intuitively, 设  $n = |Q|$ , 当我们的字符串长于  $n$  时, 前  $n + 1$  个字符共有至少  $n + 1$  种状态需要区分 (即两种字符的数量差值), 这是不可能做到的.

### 命题 1.1.3

若  $A, B$  是正则语言, 则  $A \cap B, A \cup B, \complement_{\Sigma^*} A, A \circ B, A^*$  都是正则语言.

(其中  $A \circ B := \{w = xy : x \in A \wedge y \in B\}$ ,  $A^* = \bigcup_{k=0}^{\infty} A^{\circ k}$ .)

→ Proof. 前三者都是明显的. 后两者的速通解释是: 我们可以通过有限状态的 DP (你谁?) 判断给定  $w$  是否被接受, 那么添加任意有限记录维度的但具有非确定转移的 DP 也是有限状态的, 把这些记录维度直接展开的确定转移 DP (像是 DP of DP) 的状态数也是有限的, over.

但我们为什么要展开所谓的非确定性转移呢?

### 定义 1.1.4 (NFA)

称资料  $N = (Q, \Sigma, \delta, q_0, F)$  是非确定性有限状态自动机 (NFA), 当

- $Q, \Sigma, q_0, F$  意义同上;
- $\Sigma_\varepsilon := \Sigma \sqcup \{\varepsilon\}$ ,  $\delta: Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$ .

若字符串  $w_1, \dots, w_n = w \in \Sigma^*$  在输入  $M$  后到达接受状态  $q \in F$ , 即

$$\begin{aligned}
& \exists w' \in \Sigma_\varepsilon^*, (w' \text{ is } w \text{ but some } \varepsilon \text{ inserted}) \\
& \wedge r_0 = q_0 \\
& \wedge (\exists (r_0, \dots, r_{|w'|}), \forall i \in [1 : |w'|], r_i \in \delta(r_{i-1}, w'_i)) \\
& \wedge r_{|w'|} \in F.
\end{aligned}$$

则称  $w$  被  $M$  接受.

DFA 自然地能被升级为 NFA.

### 定理 1.1.5

NFA 编码的语言都是正则语言 (即存在与之等价的 DFA).

→ *Proof.* 思路其实和上面的讨论别无二致. 对 NFA  $N$ , 容易构造  $M = (Q', \Sigma', \delta', q'_0, F')$ , 其中

- $Q' = \mathcal{P}(Q)$ ;
- $\Sigma' = \Sigma$ ;
- $\delta'(S, a) = \bigcup_{p \in S} \bigcup_{k=0}^{\infty} (\delta(\cdot, \varepsilon))^k (\delta(p, a))$ ;
- $q'_0 = \bigcup_{k=0}^{\infty} (\delta(\cdot, \varepsilon))^k (q_0)$ ;
- $F' = \{S \in Q' : F \cap S \neq \emptyset\}$ .

于是, NFA 变成了一种强大的构建 DFA 的方式. 命题 1.3 的结论都可以用 NFA 轻松编码.

## § 1.2 正则表达式

### 定义 1.2.1 (正则表达式)

字符串集合  $R$  是正则表达式, 当且仅当它一定满足如下之一:

- $R = \{a\}$ , 其中  $a \in \Sigma$ ;
- $R = \{\varepsilon\}$ ;
- $R = \emptyset$ ;
- $R = R_1 \cup R_2$ , 其中  $R_1, R_2$  是正则表达式;
- $R = R_1 R_2$ , 其中  $R_1, R_2$  是正则表达式, 我们省略了连接运算  $\circ$ ;
- $R = R_0^*$ , 其中  $R_0$  是正则表达式.

例如 (虽然是一些 corner case),  $\{111\} \circ \emptyset = \emptyset$ ,  $\emptyset^* = \{\varepsilon\}$ .

### 定理 1.2.2

一个语言是正则语言, 当且仅当它可以被一个正则表达式描述.

→ *Proof.* 右推左, 对正则表达式的定义进行结构归纳地构建 NFA 即可.

左推右, 我们需要将任意一个 DFA 转译为正则表达式, 为此, 我们引入充当媒介的 GNFA:

### 定义 1.2.3 (GNFA)

一个广义非确定性有限状态自动机是资料  $G = (Q, \Sigma, \delta, q_s, q_a)$ , 其中:

- $Q, \Sigma$  同 NFA;  $q_s$  是唯一起始状态,  $q_a$  是唯一接收状态.
- $\delta : (Q \setminus \{q_a\}) \times (Q \setminus \{q_s\}) \rightarrow \mathcal{R}$ , 其中  $\mathcal{R}$  是所有正则表达式构成的集合.

→ *Proof @ 1.2.2 Cont.* NFA 自然地可以被转译为 GNFA, 我们需要证明 GNFA 总是对应一个正则表达式. 归纳地, 对  $k := |Q| = 2$  的  $G$ , 显然成立. 对  $k > 2$ , 任选  $q_r \in Q \setminus \{q_s, q_a\}$ , 令  $G' := (Q' := Q \setminus \{q_r\}, \Sigma, \delta', q_s, q_a)$ , 我们只需要构造  $\delta'$  使得  $G'$  与  $G$  编码同一个语言.

对任意  $(q_1, q_2) \in Q' \setminus \{q_a\} \times Q' \setminus \{q_s\}$ , 简记  $R_{ij} = \delta(q_i, q_j)$  ( $i, j \in \{1, 2, r\}$ ), 直接枚举所有可能的路径:

$$\delta'(q_1, q_2) := R_{12} \cup R_{1r}R_{rr}^*R_{r2}.$$

明所欲证 (在 Rocq 中, 这四个字可以写作 `auto`).

## § 1.3 正则性判定

我们希望判定任意给定的语言是否是正则语言.

### 引理 1.3.1 (Pumping)

若  $A$  是正则语言, 则

$$\begin{aligned} \exists p \in \mathbb{N}, \forall s \in A, |s| \geq p \Rightarrow \exists x, y, z \in \Sigma^*, s = xyz \\ \wedge (\forall i \geq 0, xy^iz \in A) \\ \wedge |y| > 0 \\ \wedge |xy| < p. \end{aligned}$$

→ *Proof.* 设  $A$  可由 DFA  $M$  编码, 令  $p = |Q|$ , 任取  $s = s_1 \cdots s_n \in A, n \geq p$ , 它对应  $M$  上的游走路程  $r_1, \dots, r_{n+1} \in Q$ . 由鸽巢原理, 不妨设  $1 \leq i < j \leq p+1$  使得  $r_i = r_j$ . 令  $x = s_1 \cdots s_{i-1}$ ,  $y = s_i \cdots s_{j-1}$ ,  $z = s_j \cdots s_n$ , 明所欲证.

反之, 不满足 pumping 引理的语言一定不是正则语言. 例如:

- 对  $B = \{0^n 1^n : n \geq 0\}$ , 任意的  $p$  都被  $0^p 1^p \in B$  反驳.
  - 值得注意的是, 它的补集  $B = \{0^n 1^m : n \neq m\}$  非正则, 但取  $p = 3$  就满足引理.
- 对  $C = \{w : \#\{0 \in w\} = \#\{1 \in w\}\}$ , 上面的反例仍然适用.
- 对  $D = \{1^{n^2} : n \geq 0\}$ , 任意的  $p$  都被  $1^{p^2} \in D$  反驳.
- 对  $E = \{0^i 1^j : i > j \geq 0\}$ , 任意的  $p$  都被  $0^{p+1} 1^p \in E$  反驳 (循环节  $y$  的指数取 0 时).
- 对  $F = \{ww : w \in \{0, 1\}^*\}$ , 任意的  $p$  都被  $0^p 10^p 1 \in F$  反驳.

### 例子 1.3.2

设  $N \subset \mathbb{N}, A = \{0^n : n \in N\}, B = \{n_1 \cdots n_k : (\overline{n_1 \cdots n_k})_2 \in N\}$ , 证明或推翻:

- (a) 若  $A$  正则, 则  $B$  正则.
- (b) 若  $B$  正则, 则  $A$  正则.

→ *Solution.* (a) 正确.  $A$  满足 pumping 引理, 设 pumping 给出  $p$ . 取  $d = \text{lcm}\{1, \dots, p\}$ , 对任意  $s = 0^n, n > 2(d+p)$  时, 设 pumping 给出  $s = xyz$ , 总能调整  $|z| < p$  得到  $s = xy^k z$ , 其中  $|x| + |z| < 2p, |y| < p, |y^k| \geq 2d$ . 那么  $xy^{k-d/|y|}z \in A$ , 也即  $0^{n-d} \in A$ . 总之, 我们得到  $\exists n_0 \in \mathbb{N}, \forall k \geq n_0, 0^k \in A \Leftrightarrow 0^{k-d} \in A$  (存在最终周期性). 所以, 除了有限个短特例以外,  $s \in A$  可由  $|s| \bmod d$  确定. 容易构造对应的特判和维护二进制数模  $d$  的 NFA 来识别  $B$ . 所以  $B$  正则.

(b) 错误.  $N = \{n \in \mathbb{N} : 2 \mid \text{popcount}(n)\}$ ,  $A$  显然不能满足 (a) 中推导出的最终周期性.

## § 1.4 CFL 与 PDA

### 定义 1.4.1 (上下文无关文法)

称资料  $G = (V, \Sigma, R, S)$  是上下文无关文法, 当且仅当

- $V$  是有限的变量集;
- $\Sigma$  是有限的终结字符集,  $\Sigma \cap V = \emptyset$ ;

- $R \subset V \times (V \sqcup \Sigma)^*$  是有限的文法集.
- $S \in V$  是起始变量.

设  $u, v, w \in (V \sqcup \Sigma)^*$ . 若  $(A, w) \in R$ , 也记作  $A \rightarrow w$ , 则对任意  $u, v$  可记  $uAv \Rightarrow u w v$ . 若  $u = w$  或者存在一系列  $u = u_0, \dots, u_k = w$  使得  $u_i \Rightarrow u_{i+1}$ , 则记  $u \xRightarrow{*} w$ .  $G$  编码的语言定义为

$$A = \{w : S \xRightarrow{*} w\}.$$

例如,  $G = (\{A, B\}, \{0, 1, \#\}, R, A)$ , 其中  $R = \{(A, 0A1), (A, B), (B, \#)\}$ , 则它的语言是  $A = \{0^n \# 1^n : n \geq 0\}$ .

对语言  $A = \{0^n 1^n : n \geq 0\} \cup \{1^n 0^n : n \geq n\}$ , 我们也能设计文法  $G = (\{A, B, C\}, \{0, 1\}, R, A)$ , 其中  $R$  包含

$$A \rightarrow \varepsilon \mid 0B1 \mid 1C0, \quad B \rightarrow \varepsilon \mid 0B1, \quad C \rightarrow \varepsilon \mid 1C0.$$

为了给这类语言构造自动机, 我们可以为 NFA 配备一个 "存储设备", 例如... 一个 "栈".

#### 定义 1.4.2 (下推自动机)

称资料  $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$  是一个下推自动机 (PDA), 当且仅当

- $Q, \Sigma, q_0, F$  同理 NFA;
- $\Gamma$  是有限集, 描述栈字母表.
- $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ .

它接受  $w = w_1 \dots w_m \in \Sigma_\varepsilon^*$ , 当且仅当存在一系列状态  $s_0, \dots, s_m \in Q$  和一系列栈状态  $t_0, \dots, t_m \in \Gamma^*$ , 满足

- $s_0 = q_0, t_0 = \varepsilon$ ;
- $\forall i \in [0 : m - 1], \exists r \in \Gamma^*, \exists a, b \in \Gamma_\varepsilon, t_i = ar \wedge t_{i+1} = br \wedge (s_{i+1}, b) \in \delta(s_i, w_{i+1}, a)$ .
- $s_m \in F$ .

记号上, 我们可以在绘制 NFA 的基础上, 在转移边添加形如  $a \rightarrow b$  的标记.

当然, 你一定喜欢 PDA 的省流描述: "只是用一个栈作为内存的 streaming algorithm".

#### 定理 1.4.3

如果一个语言是上下文无关的, 则它可被 PDA 编码.

→ Proof. 设语言的文法为  $G = (V, \Sigma, R, S)$ , 为方便描述, 我们直接以 streaming algorithm 的视角来构造:

```

1 | stack.push($)
2 | stack.push(S)
3 | repeat
4 |     c ← stack.pop()
5 |     if c ∈ V then
6 |         fork ∀(c, w) ∈ R
7 |             stack.push(w)
8 |     elif c ∈ Σ then
9 |         s ← nextchar()
10 |        if c ≠ s then
11 |            return Reject
12 |        else // c = $
13 |            return Accept

```

#### 定理 1.4.4

如果一个语言由一个 PDA 编码, 则它服从某个上下文无关文法.

→ Proof. 设  $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ , 我们不妨加强对  $P$  的限制:

- 只有一个接受状态  $q_1$  (新增一个总接受状态, 由  $\varepsilon$  边连接);
- 被接受时清空了栈 (新增若干状态, 初始时压入特殊字符, 接受时先不断弹栈直到弹出特殊字符);
- 转移时, 要不只压一次栈, 要不只弹一次栈 (否则可以拆成两步转移).

接下来, 我们构造  $G = (V, \Sigma, R, S)$ ,  $V = \{A_{pq} : p, q \in Q\}$ ,  $S = A_{q_0 q_1}$ , 有三类规则:

$$\begin{aligned}
 R_1 &= \{A_{pq} \rightarrow aA_{rs}b : p, q, r, s \in Q, u \in \Gamma, a, b \in \Sigma_\varepsilon, \\
 &\quad (r, u) \in \delta(p, a, \varepsilon) \wedge (q, \varepsilon) \in \delta(s, b, u)\}, \\
 R_2 &= \{A_{pq} \rightarrow A_{pr}A_{rq} : p, q, r \in Q\}, \\
 R_3 &= \{A_{pp} \rightarrow \varepsilon : p \in Q\}, \\
 R &= R_1 \cup R_2 \cup R_3.
 \end{aligned}$$

其组合意义是自然的:

- 第一类描述一次互相匹配的压栈-弹栈:  $u$  在  $p$  因字符  $a$  压入, 在  $s$  因字符  $b$  弹出;
- 第二类描述路径的组合: 为了从  $p$  到  $q$ , 我们可以添加任意途经点  $r$ ;
- 第三类描述  $p$  可以直接到达自己.

我们来严格证明  $G$  编码了与  $P$  相同的语言. 结论由下面的 [引理 1.4.5](#) 和 [引理 1.4.6](#) 给出.

#### 引理 1.4.5

上述语境下若  $A_{pq} \xRightarrow{*} x \in \Sigma^*$ , 则在  $P$  上有  $(p, []) \xRightarrow{x} (q, [])$ . ( $[]$  表示空栈, 注意在 PDA 的定义里栈顶是序列的开头.)

→ Proof. 对  $A_{pq} \xRightarrow{*} x$  的步数  $k$  归纳. 当  $k = 1$ , 规则必须来自  $R_3$ , 则  $x = \varepsilon, p = q$ , 成立.

接着考虑从  $k$  到  $k + 1$  的归纳. 考察第一步规则, 它只能来自  $R_1$  或者  $R_2$ :

- 若第一步是  $A_{pq} \rightarrow aA_{rs}b$ , 归纳地,  $A_{rs} \xRightarrow{*} y$  带来  $P$  上的转移  $(r, []) \xRightarrow{y} (s, [])$ , 则根据定义, 存在  $u \in \Gamma$  使得

$$(p, []) \xrightarrow{a} (r, [u]) \xRightarrow{y} (s, [u]) \xrightarrow{b} (q, []).$$

- 若第一步是  $A_{pq} \rightarrow A_{pr}A_{rq}$ , 归纳地,  $A_{pr} \xRightarrow{*} y$  和  $A_{rq} \xRightarrow{*} z$  拼起来就得到

$$(p, []) \xRightarrow{y} (r, []) \xRightarrow{z} (q, []).$$

明所欲证.

#### 引理 1.4.6

上述语境下若  $P$  上有  $(p, []) \xRightarrow{x} (q, [])$ , 则  $A_{pq} \xRightarrow{*} x$ .

→ Proof. (和上一个引理如出一辙, 别听雨兔念经了.) 对  $x$  在  $P$  上的转移次数  $k$  归纳. 当  $k = 0$ ,  $p = q, x = \varepsilon$ , 规则由  $R_3$  给出.

同样考虑从  $k$  到  $k + 1$  的归纳. 考察是否存在某个中间状态的栈是空栈:

- 若存在这样的  $(r, [])$ , 设  $(p, []) \xRightarrow{y} (r, []) \xRightarrow{z} (q, [])$ , 归纳地  $A_{pr} \xRightarrow{*} y$  且  $A_{rq} \xRightarrow{*} z$ , 那么

$$A_{pq} \rightarrow A_{pr}A_{rq} \xRightarrow{*} yz = x.$$

- 否则, 必然有  $(p, []) \xrightarrow{a} (r, [u]) \xRightarrow{x'} (s, [u]) \xrightarrow{b} (q, [])$ , 归纳地  $A_{rs} \xRightarrow{*} x'$ , 那么

$$A_{pq} \rightarrow aA_{rs}b \xRightarrow{*} ax'b = x.$$

明所欲证.

当然, 作为一个 "只用单栈的在线算法", PDA 与 CFL 的表达能力亦有局限.

#### 引理 1.4.7 (Pumping)

若  $A$  是 CFL, 则

$$\begin{aligned} \exists p \in \mathbb{N}, \forall s \in A, |s| \geq p \Rightarrow \exists u, v, x, y, z \in \Sigma^*, s = uvxyz \\ \wedge (\forall i \geq 0, uv^i xy^i z \in A) \\ \wedge |vy| > 0 \\ \wedge |vxy| \leq p. \end{aligned}$$

→ Proof. 设  $A$  可由文法  $G$  编码. 设  $b = \max\{|w| : (A, w) \in R\}$ , 设从  $S$  出发形成的推导树高为  $h$ , 则最终串长不超过  $b^h$ . 另一方面, 若  $h \geq |V| + 1$ , 则至少有一个变量在叶子-根的树链中出现了至少两次. 现在取  $p = b^{|V|+1}$ , 对任意  $s \in A$ , 若  $|s| \geq p$ , 则推导出  $s$  的树高至少是  $|V| + 1$ , 因此就存在上述重复出现的变量, 取对应子树高最小的  $A$  和它子树内和它相同的  $A'$ , 满足  $S \xRightarrow{*} uAz, A \xRightarrow{*} vA'y, A' \xRightarrow{*} x$ . 不妨设  $|vy| > 0$  (否则可以精简推导树结构而不改变已有性质), 由子树高的最小性,  $A$  子树内不再有任何出现在叶子-根树链的重复变量, 因此  $A \xRightarrow{*} vxy$  满足  $|vxy| \leq p$ . 显然  $s = uvxyz$  已然满足条件.

例如,  $L = \{a^n b^n c^n : n \geq 0\}$ . 对任意  $p, a^p b^p c^p$  都无法拆分.

注意这个  $L$  是非常典型的非 CFL 例子, 它可以用来构造很多反例. 例如:

#### 命题 1.4.8

(a) 存在 CFL  $A, B$ ,  $A \cap B$  不是 CFL.

(b) 存在 CFL  $A, \overline{A}$  不是 CFL.

→ Proof. (a)  $A = \{a^n b^n c^* : n \geq 0\}, B = \{a^* b^n c^n : n \geq 0\}, A \cap B = L$ .

(b) 其实可以作为 (a) 的平凡推论: 若  $\overline{A}$  和  $\overline{B}$  都是 CFL, 而 CFL 显然又对  $\cup$  封闭, 所以  $\overline{\overline{A \cup B}} = A \cap B$  是 CFL, 矛盾.

既然 NFA 等价于 DFA, 这个 PDA 是否等价于确定性 PDA (DPDA) 呢?

### 定理 1.4.9

存在 CFL  $A$  不是 DCFL, 即它不能被任何 DPDA 识别.

→ *Proof.* 一个漂亮的 RAA 风味证明: DCFL 显然对取补集封闭, 则 命题 1.4.8(a) 的反例  $A, B, \overline{A}, \overline{B}$  中至少有一个不是 DCFL.

构造主义者吓哭了, 所以我们给两个例子:  $\{ww^R : w \in \{0, 1\}^*\}$  和  $\{0^n 1^n : n \geq 0\} \cup \{0^n 1^{2n} : n \geq 0\}$  (免责声明: 均来自 LLM). 至于它俩是不是真的反例, 我们应该留给构造主义者自己去证.

## 第二章 图灵机

(我习惯不音译人名, 但 Turing 出现得实在太多了, 每次插一个单词反而很奇怪.)

### § 2.1 确定性图灵机

#### 定义 2.1.1 [图灵机 (Turing Machine, TM)]

(a) 一个  $k$ -tape TM 是七元组  $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ , 其中:

- $\Gamma$  是有限集, 是纸带字母表; 这里我们采用一种更方便的变种定义:
  - $\_ \in \Gamma$ , 它是唯一可以在纸带上出现无穷次的空白字符;
  - $\triangleright \in \Gamma$ , 标识输入字符串起点.
- $\Sigma \subset \Gamma \setminus \{\_ \}$  是输入字母表 (可以出现在初始纸带的字符集).
- $Q$  是有限的状态集 (注意它不保存任何读写头位置信息和纸带信息).
- $q_0 \in Q$  是初始状态;  $q_{\text{accept}}$  是接受状态;  $q_{\text{reject}}$  是拒绝状态.
- $\delta : (Q \setminus F) \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, N, R\}^k$  是状态转移函数.

(b) TM 运行时, 设当前状态  $q$ , 读写头读入纸带符号  $x = (x_1, \dots, x_k)$ , 则取转移  $\delta(q, x) = (q', x', z)$ , 其中:

- $q'$  描述新状态;
- $x'$  描述当前读写头的写入信息;
- $z$  描述每个读写头的移动方式.

(c) TM 的格局 (configuration 感觉翻译成构型会更高级)  $C$  记录了其某一时刻的状态, 纸带完整信息和读写头位置. 称 TM  $M$  输入  $w$  时停机, 若存在一系列格局  $C_0, \dots, C_t$ , 使得

- $C_0$  是起始格局:  $q = q_0$ , 所有读写头在各自纸带最左侧;
- $C_i$  可由  $C_{i-1}$  一步转移到, 且  $C_i$  ( $i < t$ ) 的状态均不是停机状态;
- $C_t$  的状态为停机状态  $q_{\text{accept}}$  (被接受) 或  $q_{\text{reject}}$  (被拒绝).

#### 约定 2.1.2

此后我们简记 (注意这些记号并不通用于其他资料):

- $M \xrightarrow{w} A$ ,  $M$  接受  $w$ ;  $M \xrightarrow{w} R$ ,  $M$  拒绝  $w$ .
- $M \xrightarrow{w} H$ ,  $M$  输入  $w$  会停机;  $M \not\xrightarrow{w} H$ ,  $M$  输入  $w$  不停机.



- $M[w]$ : 已填充输入为  $w$  的 TM (一台新图灵机).
- $M(w) = x: M \xrightarrow{w} H$  且按照特定规则能够在输出纸带读出  $x$ .
  - 特别地若  $M[w] \circlearrowleft$  且  $M'[w'] \circlearrowleft$ , 我们也认为  $M(w) = M'(w')$ .

相比上面的各类自动机, TM 的结构更复杂, 其运行的 "结果" 也更复杂: 有接受, 拒绝和不停机这三种可能. 为此, 我们需要更细致地规定其描述的语言:

### 定义 2.1.3

(a) 称语言  $A$  是图灵可识别 (Turing-recognizable) / 递归可枚举 (recursively enumerable) 的, 当且仅当存在 TM  $M$  使得

$$\left( \forall w \in A, M \xrightarrow{w} A \right) \wedge \left( \forall w \in \overline{A}, M \xrightarrow{w} R \vee M \circlearrowleft^w \right).$$

记  $L(M) = A$ .

(b) 称语言  $A$  是图灵可判定 (Turing-decidable) 的, 当且仅当存在 TM  $M$  使得

$$\left( \forall w \in A, M \xrightarrow{w} A \right) \wedge \left( \forall w \in \overline{A}, M \xrightarrow{w} R \right).$$

这样永不死循环的  $M$  称为判定机 (decider).

喜闻乐见地, 我们也开始关注这台大机器的计算效率:

### 定义 2.1.4 (计算, 计算复杂度, 时间可构造函数)

- (a) 对  $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ , 称  $M$  计算  $f$ , 当且仅当  $\forall x \in D_f, f(x) = M(x)$ .
- (b) 称  $M$  在  $T(n)$  运行, 当且仅当对任意输入  $x$ ,  $M[x]$  在不超过  $T(|x|)$  步停机.
- (c) 称  $T: \mathbb{N} \rightarrow \mathbb{R}$  是时间可构造 (time-constructible) 的, 当且仅当存在 TM 计算  $x \mapsto \lfloor T(|x|) \rfloor$ .

来一点简单的例子吧! 我们先装作不知道 TM 的强大能力, 只在原本定义下构造结果.

例:  $A = \{w\#w : w \in \{0, 1\}^*\}$  可以被 1-tape TM  $\mathcal{O}(n^2)$  地识别:

- 每次读入第一个非  $\triangleright$  字符  $x$ .
- 循环右移, 直到经过  $\#$  且当前字符  $y \neq \triangleright$ :
  - 若  $x = \_$ , 对比完成, 接受; 若  $x \neq y$ , 拒绝.
  - 否则  $x = y$ ,  $y$  写为  $\triangleright$ .
- 循环左移, 直到回到  $x$  的位置, 将  $x$  写为  $\triangleright$ .

例:  $A = \{ww : w \in \{0, 1\}^*\}$  可以被 2-tape TM  $\mathcal{O}(n)$  地识别:

- 循环地将整个输入串复制到纸带 2.
- 两个读写头  $p_1, p_2$  移动回各自纸带的字符串开头. 接着循环移动 (Floyd 判环的 trick):
  - $p_2$  右移一次.
  - $p_1$  右移两次, 若第一次右移后读到  $\_$ , 拒绝; 若第二次右移读到  $\_$ , 跳出循环.
- $p_1$  移动回起点. 接着循环移动:
  - 若  $p_2$  读到  $\_$ , 接受; 否则若  $p_1$  和  $p_2$  读入字符不同, 拒绝.

- $p_1, p_2$  各自右移一次.

没听说过 TM 还能用超大有限字符集? 没听说过 TM 还能用多纸带? 没关系, 它们都不会强于我们印象中的原味 TM.

### 定理 2.1.5

对任意 TM  $M$ , 若它在  $\Gamma$  工作, 于  $T(n)$  计算  $f: \{0, 1\}^* \rightarrow \{0, 1\}$ , 则存在 TM  $M'$ , 它在  $\Gamma' = \{0, 1, \triangleright, \_ \}$  工作, 于  $\mathcal{O}(T(n) \log |\Gamma|)$  计算  $f$ .

→ *Proof.* 把每个符号展开成  $\mathcal{O}(\log |\Gamma|)$  个 bit, 用相应的时间模拟原 TM 的单步转移.

### 定理 2.1.6

对任意  $k$ -tape TM  $M$ , 若它于  $T(n)$  计算  $f: \{0, 1\}^* \rightarrow \{0, 1\}$ , 则存在 1-tape TM  $M'$  于  $\mathcal{O}(kT(n)^2)$  计算  $f$ .

→ *Proof.* 交错编码  $k$  条纸带即可.

### 论点 2.1.7 (Church-Turing)

所有直观意义下的算法都能编码到图灵机.

我们做证明的时候都已经相信过这一点啦.

### 引理 2.1.8

存在嵌入  $\mathcal{D}: \{M: \text{TM}\} \hookrightarrow \mathbb{N}$ , 且计算它的代价足够低.

这似乎是一个明显到无人在意但确实很有奠基性的引理. (如果你把  $\mathcal{D}$  当成反汇编器,  $\mathcal{D}^{-1} := \mathcal{C}$  就是编译器.)

为了方便后面的证明, 若  $\mathcal{D}^{-1}(\alpha) = \emptyset$ , 约定  $\mathcal{C}(\alpha)$  对应一个直接拒绝的 TM.

### 定理 2.1.9

存在通用 (universal) 图灵机 (UTM)  $U$ , 使得  $\forall x, \alpha \in \{0, 1\}^*$ ,  $U(x, \alpha) = M_\alpha(x)$ , 其中  $M_\alpha := \mathcal{D}^{-1}(\alpha)$ , 且若  $M_\alpha[x]$  用  $T$  步停机, 则  $U[x, \alpha]$  用  $CT \log T$  步停机, 其中  $C$  是只与  $M_\alpha$  有关的参数.

→ *Proof.* 利用 定理 2.1.6, 将  $M_\alpha$  转化为 1-tape TM  $M$ , 则  $M$  用  $\mathcal{O}(T^2)$  步停机. 我们构造  $U$  为一个 5-tape TM, 其中各个纸带的功能为:

1. 记录输入;
2. 记录  $M$  的描述;
3. 对  $M$  的模拟;
4.  $M$  的状态;
5. 记录输出.

沿此思路做一些细节构造即可, 在此略过 (因为 ck 也略过了).

*Remark.*

我也想到过这个问题, 或许你也发现证明的实例就在身边: 看看你的操作系统呢?

## § 2.2 可计算性

在定义 2.1.3 中, 我们知道

$$\{\text{decidable languages}\} \subset \{\text{recognizable languages}\} \subset \{\text{languages}\}.$$

它们是否都是真包含关系? 能否找到各自的例子? 如何理论描述这些问题?

我已在数分, 高代, 离散和 TOC 四门课听了四次集合论入门.

以后默认  $\Sigma = \{0, 1\}$ .

### 定理 2.2.1

存在不可识别的语言  $A \subset \Sigma^*$ .

→ Proof. 引理 2.1.8 给出  $|\{\text{TMs}\}| = \aleph_0 < |\Sigma^*| = \aleph_1$ .

构造主义者就要闹了, 我们不理他就好.

### 定理 2.2.2

存在不可判定的语言  $A \subset \Sigma^*$ .

→ Proof. 为了安抚构造主义者, 我们构造函数

$$UC : \Sigma^* \rightarrow \Sigma, \alpha \mapsto 1 - [\mathcal{C}(\alpha) \xrightarrow{\alpha} A].$$

我们断言这个 UC 是不可计算的, 也即  $f^{-1}(1) \subset \Sigma^*$  不可判定.

反证, 若有 TM  $M$  计算 UC, 应当有  $M(\mathcal{D}(M)) = UC(\mathcal{D}(M))$ , 但是根据 UC 的定义,  $UC(\mathcal{D}(M)) = 1 - M(\mathcal{D}(M))$ , 矛盾.

可以看出, UC 的构造其实来源于经典的 Cantor 对角线法. 而它的一个更著名的推论是停机问题的不可计算性.

### 推论 2.2.3

(a) 语言  $\text{HALT} = \{\alpha \# x : \mathcal{C}(\alpha) \xrightarrow{x} H\}$  不可判定.

(b) 语言  $A_{\text{TM}} = \{\alpha \# x : \mathcal{C}(\alpha) \xrightarrow{x} A\}$  不可判定但可识别.

→ Proof. (a) 如果存在  $M_{\text{HALT}}$  判定 HALT, 构造判定机  $M$ :

$$M(\alpha) = \begin{cases} 1 - \mathcal{C}(\alpha)(\alpha), & M_{\text{HALT}}(\alpha \# \alpha) = 1; \\ 1, & \text{otherwise.} \end{cases}$$

它计算了 UC, 但这不可能.

(b)  $A_{\text{TM}}$  显然可识别: 直接 compile & run 即可. 但如果存在  $M_A$  判定  $A_{\text{TM}}$ , 构造判定机  $M$ :

$$M(\alpha) = 1 - M_A(\alpha \# \alpha).$$

它又计算了 UC, 这不可能.

这个推论的证明思路就是我们常说的归约方法. 我们还能再语言上更严格地描述这种归约关系:

### 定义 2.2.4 [映射可归约性 (mapping reducibility)]

称语言  $A$  映射可归约到  $B$ , 写作  $A \leq_m B$ , 当且仅当

$$\exists (\text{computable } f : \Sigma^* \rightarrow \Sigma^*, \forall w \in \Sigma^*, (w \in A \Leftrightarrow f(w) \in B)).$$

显然, 如果  $A \leq_m B$ , 那么:

- 若  $B$  可判定, 那么  $A$  可判定 (先计算  $f$  再在  $B$  上判定);
- 若  $A$  不可判定, 则  $B$  不可判定 (否则先计算  $f$  再在  $B$  上判定就判定了  $A$ ).

所以, 为了证明某个语言  $B$  不可判定, 我们只需要找到一个已知不可判定的  $A$  并证明  $A \leq_m B$ . 推论 2.2.3 便是将 UC 作为这样的  $A$  的例子.

### 例子 2.2.5

(a) 语言  $E_{TM} = \{\alpha : \nexists w \in \Sigma^*, C(\alpha) \xrightarrow{w} A\}$  不可判定.

(b) 语言  $EQ_{TM} = \{\alpha \# \beta : L(C(\alpha)) = L(C(\beta))\}$  不可判定.

→ *Proof.* (a) 考虑归约  $A_{TM} \leq_m \overline{E_{TM}}$ . 若有  $M_E$  判定后者, 则构造  $M$ :

$$M(\alpha \# x) = M_E(\beta_{\alpha, x}),$$
$$C(\beta_{\alpha, x})(y) = \begin{cases} 0, & x \neq y; \\ C(\alpha)(y), & x = y. \end{cases}$$

它就判定了  $A_{TM}$ , 矛盾.

(b) 取  $C(\beta) \rightarrow R$  即可, 这样  $EQ_{TM}$  的判定器可以判定  $E_{TM}$ .

我们再来研究刚刚提到的五个语言 (UC, HALT,  $A_{TM}$ ,  $E_{TM}$ ,  $EQ_{TM}$ ) 的可识别性:

- $\overline{E_{TM}}$  可识别: 设  $\Sigma^* = \{x_1 \prec x_2 \prec \dots\}$ , 接着运行  $C(\alpha)[x_1]$  一步, 再运行  $C(\alpha)[x_1]$  和  $C(\alpha)[x_2]$  各一步, ..., 如此可以在有限步内发现接受者.
- $A_{TM}$  可识别: 直接 compile & run.
- HALT 可识别: 直接 compile & run.
- $\overline{UC}$  可识别: 直接 compile & run.

我们不太容易严格论证其他的语言不可识别, 但这些观察也启发我们了如下结论:

### 定理 2.2.6

语言  $A$  可判定, 当且仅当  $A$  和  $\overline{A}$  可识别.

→ *Proof.* 左推右显然. 右推左, 交替运行两个识别器, 总有一个在有限步内返回.

因此  $E_{TM}$ ,  $\overline{A_{TM}}$ ,  $\overline{HALT}$  和 UC 都不可识别. 还剩下  $EQ_{TM}$  和  $\overline{EQ_{TM}}$ , 我们尝试证明二者皆不可识别:

### 命题 2.2.7

$EQ_{TM}$  和  $\overline{EQ_{TM}}$  不可识别.

→ *Proof.* 对于  $EQ_{TM}$ , 若  $M_Q$  识别它, 则构造  $M$  满足

$$M(\alpha \# x) = M_Q(\alpha \# \beta_{\alpha, x}),$$
$$C(\beta_{\alpha, x})(y) = \begin{cases} C(\alpha)(y), & x \neq y; \\ 0, & x = y. \end{cases}$$

则  $M$  识别了 " $C(\alpha)$  不接受  $x$ ", 即  $\overline{A_{TM}}$ , 矛盾.

对于  $\overline{EQ_{TM}}$ , 若  $M'_Q$  识别它, 完全对称地构造  $M'$  满足

$$M'(\alpha \# x) = M'_Q(\alpha \# \beta_{\alpha, x}),$$

$$C(\beta_{\alpha, x})(y) = \begin{cases} C(\alpha)(y), & x \neq y; \\ 1, & x = y. \end{cases}$$

则  $M'$  识别了 " $C(\alpha)$  不接受  $x$ ", 也即  $\overline{A_{TM}}$ , 矛盾.

下面两个命题来自 第二次作业 7.

### 引理 2.2.8

给定 1-tape TM  $M$  和输入  $w$ , 存在对  $M[w]$  运行历史的编码方式, 设  $A = \{H : M \text{ correctly accepted } w \text{ in } H\}$  包含所有正确记录  $M$  接受  $w$  的运行历史的字符串 (至多一个元素), 则  $\overline{A}$  是 CFL.

→ *Proof.* 对一系列格局  $C_0, \dots, C_k$ , 构造其编码  $\mathcal{H}(C_0, \dots, C_k) = \#q_0@w_0\#q_1@w_1^R\#\dots\#$ , 其中  $q_i$  是  $C_i$  的 TM 状态编码,  $w_i$  是  $C_i$  的纸带状态, 并包括了特殊字符  $\uparrow$  记录指针位置. 构造 PDA  $P$ , 它接受所有  $\overline{A}$  内的字符串. 具体地,  $P$  非确定性地选择如下错误原因之一:

**分隔符错误:** 字符串的开头或结尾不是  $\#$ , 或者两个相邻的  $\#$  之间包括零个或多个  $@$ , 或者存在相邻  $\#@$ ,  $@\#$ ,  $\#\#$ .

**初始格局错误:** 读取前两个  $\#$  之间的编码  $q_0@w_0$ , 字符串比对它是否和期待的  $C_0$  一致.

**接受格局错误:** 非确定性地选取一对相邻的  $\#$  读取  $q@w$  (或  $q@w^R$ , 可以在 PDA 内部维护当前  $w$  是否反转), 如果发现这不是最后一个格局, 直接拒绝; 否则检查  $q$  是否为  $q_{\text{accept}}$ .

**状态转移错误:** 非确定性地读取三个相邻  $\#$  之间的  $q@w^R$  和  $(q')@(w')$  (后一项反转时完全同理), 不妨设  $|w| = |w'|$ : 先读取并记录  $q$ , 再读取  $w^R$  并逐字符压栈; 接着读取并记录  $q'$ , 然后读取  $w'$  并弹栈, 非确定性地枚举出指针附近 (包括指针下一个字符, 指针移动到的位置) 的字符串, 这个小子段中只有有限种可能, 因此可以在 PDA 中硬编程实现 TM 转移状态  $q'$ , 纸带状态  $w'$  和指针移动情况的检查. 其他情况要求来自  $w^R$  的弹出字符与来自  $w'$  的输入字符保持一致即可.

综上,  $P$  识别了  $\overline{A}$ ,  $\overline{A}$  是 CFL.

这个引理给出如下直接推论:

### 命题 2.2.9

$\text{ALL}_{\text{CFG}} = \{\alpha : C(\alpha) \text{ defined a CFG} \wedge L(C(\alpha)) = \Sigma^*\}$  不可判定.

→ *Proof.* 利用 引理 2.2.8 归约到  $A_{TM}$ .

*Remark.*

好啦, 我们似乎已经对图灵可识别性做出了有一定见地的刻画, 但也可以隐约察觉图灵可识别语言与所有语言这两个集合之间的巨大空洞... 我们要加强图灵机! 为了方便自然语言叙述, 我们就从程序语言的角度来看这些机器.

一个自然的想法是, 我们允许代码长度 (可数) 无穷. 但这太过平凡: 叠加可数无穷条 if 语句就能识别所有语言.

那么... 如果我们允许图灵机完成超任务呢? 具体来说, 我们可以加入一条指令

`syscall0 M x`

它以参数  $x$  调用 TM  $M$  (可以视作编码在程序里的一个函数), 并立即获知  $M[x]$  的结果, 包括停机时的输出或者 " $M[x]$  不停机" 这个事实. (其实我也想把这条指令叫做 **dream**, 表示托梦.)

我们称原本的图灵机为 0-TM, 记为  $\mathcal{M}_0$ , 允许 syscall  $k$ -TM 的 "TM" 称为  $(k+1)$ -TM, 记为  $\mathcal{M}_{k+1}$ , 对应的 syscall 指令记为 **syscall** <sup>$k$</sup> . 明显,  $\mathcal{M}_{k+1}$  总是严格强于  $\mathcal{M}_k$ , 但由于  $|\mathcal{M}_k| = \aleph_0$ , 它们都无法识别所有语言, 但也因此我们可以像研究 TM 一样研究这些 "高阶 TM" 的性质.

再进一步, 我们或许可以构造 " $\omega$  层嵌套 syscall 的  $\mathcal{M}_\omega$ ", 等等. (之所以需要区分 syscall 阶次, 是因为我觉得需要避免 " $M$  syscall  $M$ " 这样的自指行为, 这样的行为很可能让系统不一致.)

我暂时没法给出这些高阶 TM 的有趣性质, 但这个定义本身也很有趣不是吗~

## 第三章 图灵机时间复杂性

### § 3.1 时间复杂性

#### 定义 3.1.1

对时间函数  $T: \mathbb{N} \rightarrow \mathbb{N}$ ,  $\text{DTIME}(T) := \{A \in \Sigma^* : A \text{ decidable in } T(n)\}$ .

#### 定理 3.1.2 [时间谱系 (time hierarchy)]

若  $g$  是时间可构造函数,  $f(n) \log f(n) = o(g(n))$ , 则  $\text{DTIME}(f(n)) \subsetneq \text{DTIME}(g(n))$ .

→ *Proof*. 只需说明不等号. 构造语言  $E$  的判定机  $M_E$ :

- 输入  $\alpha$ , 模拟  $\mathcal{C}(\alpha)[\alpha]$  运行  $f'$  步, 其中  $f' \log f' = \Theta(g)$ .
- 如果得到了输出  $b$ , 则输出  $1 - b$ ; 否则输出 1.

计数器和 TM 模拟的每步需要  $\log f'$ , 所以  $M_E$  是  $\mathcal{O}(f' \log f')$  的. 如果  $E \in \text{DTIME}(f)$ , 则存在判定机  $M^*$  于  $\mathcal{O}(f)$  判定  $E$ .

作为判定机,  $M^*(\mathcal{D}(M^*)) = [\mathcal{D}(M^*) \in E]$ ; 但  $1 - M_E(\mathcal{D}(M^*)) = [\mathcal{D}(M^*) \in E]$ , 不可能超时. 得到矛盾.

#### 定义 3.1.3

(a) 时间复杂性类  $\mathbf{P} := \bigcup_{c \geq 0} \text{DTIME}(n^c)$ .

(b) 语言  $L$  属于时间复杂性类  $\mathbf{NP}$ , 当且仅当

$$\exists \text{ poly } P: \mathbb{N} \rightarrow \mathbb{N}, \exists M \in \mathbf{P}, \forall x \in \Sigma^*, \left( x \in L \Leftrightarrow \exists u \in \Sigma^{P(|x|)}, M(x \# u) = 1 \right).$$

即, 存在一个 take 多项式长度 "证书" 的验证机在多项式时间验证是否  $x \in L$ .

(c) 时间复杂性类  $\mathbf{EXP} := \bigcup_{c \geq 0} \text{DTIME}(2^{n^c})$ .

独立集问题 ( $u$  编码取出的独立集), 旅行商问题 ( $u$  编码一条具体路径), 子集和问题 ( $u$  编码取出的元素列表) 都是典型的  $\mathbf{NP}$  问题.

#### 命题 3.1.4

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP}.$$

→ *Proof*. 前一对显然, 后一对直接枚举证书  $u$  即可.

根据 定理 3.1.2,  $\mathbf{P} \subsetneq \mathbf{EXP}$ , 所以  $\mathbf{P} \subseteq \mathbf{NP}$  和  $\mathbf{NP} \subseteq \mathbf{EXP}$  至少一个不取等. 有生之年会知道是哪个吗?

#### 定义 3.1.5 [非确定性图灵机 (Non-deterministic Turing Machine, NDTM)]

(a) 简略地说, 在 TM 的基础上, NDTM 的  $\delta$  可以有多个 (WLOG, 两个) 输出, 分支出多个并行运行的 NDTM. 它接受某个输入, 当且仅当存在任何一个分支接受.

(b) 称 NDTM  $N$  于  $T(n)$  运行, 当且仅当对所有长为  $n$  的输入,  $N$  的所有分支都在  $T(n)$  步内停机.

(c) 对  $T: \mathbb{N} \rightarrow \mathbb{N}$ ,  $\text{NTIME}(T)$  表示所有被某个 NDTM  $\mathcal{O}(T(n))$  判定的语言.

### 定理 3.1.6

设  $\text{NP}' = \bigcup_{n \geq 0} \text{NTIME}(n^c)$ , 则  $\text{NP}' = \text{NP}$ .

→ *Proof.* 右含于左, 以 NDTM  $\mathcal{O}(P)$  地并行枚举出所有可能的证书, 再在每个分支  $\mathcal{O}(P)$  验证即可.

左含于右, 把每次非确定性选择的正确路径记录在证书里, 交由 TM 验证.

为了探索 **P** 和 **NP** 之间的关系, 我们可以先尝试找到 **NP** 中最 "难" 的问题, 也即建立 **NP** 中的难度关系.

### 定义 3.1.7 (多项式归约, NP-hard, NP-complete)

(a) 称  $A \leq_p B$ , 当且仅当存在多项式可计算的  $f: \Sigma^* \rightarrow \Sigma^*$ , 使得  $\forall x \in \Sigma^*, x \in A \Leftrightarrow f(x) \in B$ .

(b) 若  $\forall A \in \text{NP}, A \leq_p L$ , 则称  $L$  是 **NP-hard** 的.

(c) 若  $L \in \text{NP}$  是 **NP-hard** 的, 则称  $L$  是 **NP-complete** 的.

如果 **NP-complete** 的  $L$  存在, 那么  $L \in \text{P} \Leftrightarrow \text{P} = \text{NP}$ , 找到一个  $L$  的例子无疑是重要的.

### 定理 3.1.8 (Cook-Levin)

设  $\varphi$  为 CNF Boolean 表达式, 即  $\varphi = \bigwedge_i \bigvee_j v_{ij}$ , 其中  $v_{ij} \in \{x_n\} \cup \{\overline{x_n}\}$ . 则

(a) 设  $\text{CKT-SAT} = \{\text{all satisfiable CKT formulas}\}$ , 它是 **NP-complete** 的. 其中 CKT 是任意多项式层的逻辑函数集.

(b) 设  $\text{SAT} = \{\text{all satisfiable CNF formulas}\}$ , 它是 **NP-complete** 的;

(c) 设  $^3\text{SAT} = \{\text{all satisfiable 3-CNF formulas}\}$ , 它是 **NP-complete** 的.

→ *Proof.* (a) **CKT-SAT** 的 **NP-complete** 性是容易的: 它显然在 **NP** 中, 且通过硬编码验证机的格局转移就能构造相应电路. (或者, 把你电脑的时序逻辑展开成多项式层组合逻辑!)

(b) 从 (a) 出发, 要求证书提供  $u$  和所有 (a) 中电路所有结点的输出, 容易用 CNF 判断电路元件合法性.

(c) 对 (b) 的任意子句  $C_i = A_i \vee B_i$ , 其中  $B_i$  有两个字符, 则新建变量  $u_i$ ,  $D_i = A_i \vee u_i$ ,  $D'_i = B_i \vee \neg u_i$ , 最后将所有  $D$  字句合取即可.

*Remark.*

关于 (b), 我觉得上面的证法比课上那堆编码 TM 的 dirty work 聪明许多.

## § 3.2 时间完全性问题

定理 3.1.8 给出了三个 **NP-complete** 问题, 接下来喜闻乐见的工作便是向它的归约.

以下问题都是 **NP-complete** 的:

### 命题 3.2.1 (最大独立集)

INDSET: 给定无向图  $G$  和整数  $k$ , 判定  $G$  中是否存在大小为  $k$  的独立集.

→ *Proof.* 显然  $\text{INDSET} \in \text{NP}$ . 接下来尝试证明  $^3\text{SAT} \leq_p \text{INDSET}$ . 构造多项式可算的

$$f: \psi \rightarrow (G, k), \quad \psi \in ^3\text{SAT} \Leftrightarrow (G, k) \in \text{INDSET}.$$



对  $\psi$  中每个字句  $C_i$ , 共有 (不超过) 7 种赋值使它 SAT, 为每种赋值创建一个顶点. 则  $|V| = 7m$ . 如果不同  $C_i$  的两个赋值不相容, 则建一条边. 后续就简单了.

### 命题 3.2.2 (最小点覆盖)

VTXCOV: 给定无向图  $G$  和整数  $k$ , 判定  $G$  中是否存在  $k$  个结点覆盖了所有边.

→ *Proof.* 显然  $\text{VTXCOV} \in \mathbf{NP}$ , 而  $\text{INDSET} \leq_p \text{VTXCOV}$ , 因为点覆盖的补是独立集.

### 命题 3.2.3 (整数规划)

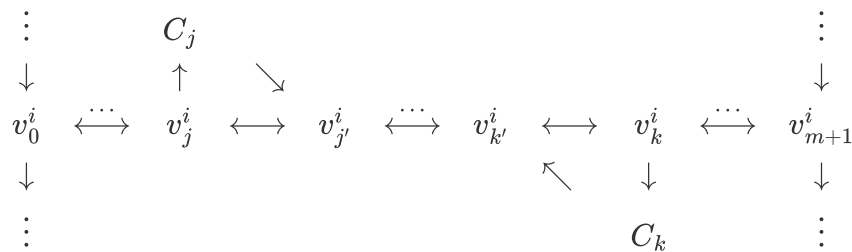
IPOG: 给定  $m$  个有理系数线性不等式, 判定是否存在对  $x \in \{0, 1\}^n$  的赋值使所有不等式成立.

→ *Proof.* 显然  $\text{IPOG} \in \mathbf{NP}$ .  $^3\text{SAT} \leq_p \text{IPOG}$  的方法比较多, 比如暴力把  $a \vee b \vee c$  记为  $a + b + c \geq 1$ .

### 命题 3.2.4 (Hamilton 路)

HPATH: 给定有向图  $G$ , 判定  $G$  中是否存在 Hamilton 路 (点不交地遍历所有点的路径).

→ *Proof.* 是一个用 SAT 的巧妙归约构造. 对某个变量  $u_i$  设它在某个子句  $C_j$  中取  $u_i$ , 在另一个  $C_k$  中取  $\neg u_i$ , 构造形如:



(我们就免去唠叨地严谨叙述了.) 可见, 我们通过  $v^i$  这条链的遍历方向来规定了  $u_i$  的真值, 当  $u_i$  SAT 了某个合取子式, 它就可以顺向进入子式结点.

### 命题 3.2.5 (Hamilton 环路)

HCCYCLE: 给定有向图  $G$ , 判定  $G$  中是否存在 Hamilton 环路.

→ *Proof.* 新增  $v_s$  连向所有原图点, 所有原图点连向  $v_t$ ,  $v_t$  连向  $v_s$ .

### 命题 3.2.5 (无向 Hamilton 路)

uHCCYCLE: 给定无向图  $G$ , 判定  $G$  中是否存在有向 Hamilton 路.

→ *Proof.* 用有向版本归约. 在有向图中, 把每个结点拆为  $x, y, z$  三个点, 连接  $(x, y), (y, z)$ , 有向边  $(u, v)$  构造为  $(u_z, v_x)$  即可.

### 命题 3.2.5 (旅行商问题)

TSP: 给定带权有向图  $G$ , 判定  $G$  存在访问每个结点恰好一次的路径, 使得边权和不大于  $k$ .

→ *Proof.* 用 HPATH 容易归纳. 环路版本也一样.

## § 3.3 NP 相关的复杂性类

### 定义-命题 3.3.1

(a)  $\text{coNP} := \{L : \neg L \in \mathbf{NP}\}$ .

(b)  $L \in \text{coNP}$  当且仅当



$$\exists \text{ poly } P : \mathbb{N} \rightarrow \mathbb{N}, \exists M \in \mathbf{P}, \forall x \in \Sigma^*, \left( x \in L \Leftrightarrow \exists u \in \Sigma^{P(|x|)}, M(x\#u) = 0 \right).$$

两个定义等价.

注意  $\mathbf{coNP} \neq \neg \mathbf{NP}$ . 事实上  $\mathbf{P} \subset \mathbf{NP} \cap \mathbf{coNP}$ . 我们也能研究所谓 " $\mathbf{coNP}$ -complete" 的问题.

$\mathbf{coNP} \stackrel{?}{=} \mathbf{NP}$  也是一个 open problem. 例如,  $\neg \mathbf{SAT}$  是  $\mathbf{coNP}$ -complete 的,  $\mathbf{coNP} = \mathbf{NP}$  意味着我们能够给出一个多项式长度的 "说明一个 SAT 问题不可满足" 的证书, 人们对此大多不太有信心.

### 定义 3.3.2

$$\mathbf{NEXP} := \bigcup_{c \geq 0} \mathbf{NTIME}(2^{n^c}).$$

### 定理 3.3.3

$$\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{EXP} = \mathbf{NEXP}.$$

→ Proof. 考虑某个  $L \in \mathbf{NTIME}(2^{n^c})$ , 它被 NDTM  $N$  判定, 设  $L' := \{x\#1^t : t = 2^{|x|^c}\}$ , 那么  $x \in L \Leftrightarrow x\#1^t \in L'$ . 根据定义,  $L' \in \mathbf{NP}$ , 如果  $\mathbf{P} = \mathbf{NP}$ , 有  $L' \in \mathbf{P}$ , 反过来就给出  $L \in \mathbf{EXP}$ .

Remark.

一个比较令人(我)满意的 happy ending 是  $\mathbf{P} = \mathbf{NP}$ , 但依赖 RAA. 当然这方面的畅想也会引出很多有意思的脑洞问题.

## 第四章 图灵机空间复杂性

### § 4.1 空间复杂性

#### 定义 4.1.1

函数  $S : \mathbb{N} \rightarrow \mathbb{N}$  描述在输入规模为  $n$  时, 某台 TM 最大的 work tape 使用量 (work tape 排除了 input tape, 这时通常认为 input tape 不可写). 一般来说  $S$  是空间可构造 (space-constructible) 的, 即  $S(n)$  可在  $\mathcal{O}(S(n))$  空间计算.

称  $L \in \mathbf{SPACE}(S(n))$ , 当且仅当存在  $\mathcal{O}(S(n))$  的 TM  $M$  判定  $L$ ; 称  $L \in \mathbf{NSPACE}(S(n))$ , 当且仅当存在  $\mathcal{O}(S(n))$  (指所有分支停机且 work tape 用量不超过这一量级) 的 NDTM  $N$  判定  $L$ .

不同于时间复杂性一般考虑的  $T(n) \geq n$ , 由于假定输入纸带不可写, 我们更感兴趣  $S(n) < n$  的问题. 而为了 TM 至少能描述输入串的下标, 通常也考虑  $S(n) \geq \log n$ .

$$\mathbf{DTIME}(S(n)) \subset \mathbf{SPACE}(S(n)) \subset \mathbf{NSPACE}(S(n)) \subset \mathbf{DTIME}(2^{\mathcal{O}(S(n))}).$$

→ Proof. 只说明最后一个包含:  $\mathbf{NSPACE}(S(n))$  的 NDTM 只产生  $2^{\mathcal{O}(S(n))}$  种格局, 在格局图上搜索是否存在从初始格局到接受格局的路径, 就能模拟一台 NDTM.

#### 定义 4.1.3

定义空间复杂性类:

- $\mathbf{PSPACE} := \bigcup_{c \geq 0} \mathbf{SPACE}(n^c)$ ;
- $\mathbf{NPSPACE} := \bigcup_{c \geq 0} \mathbf{NSPACE}(n^c)$ ;
- $\mathbf{L} := \mathbf{SPACE}(\log n)$ ;
- $\mathbf{NL} := \mathbf{NSPACE}(\log n)$ .

例如,  $\mathbf{SAT} \in \mathbf{PSPACE}$ , 同理用证书版本的定义可知  $\mathbf{NP} \subset \mathbf{PSPACE}$ .

一些 tricky 的小例子:

- $\text{EVEN} := \{x : x \text{ has even \# of } 0\} \in \text{SPACE}(1)$ , 当然你也可以认为它是不需要任何空间的自动机.
- $\text{MULT} := \{(a, b, ab)\} \in \mathbf{L}$ , 我们可以先枚举位 ( $\log n$  space), 再枚举检查所有贡献到这一位的 1, 检查奇偶性是否与  $ab$  中的对应位置相同.
- $\text{PATH} := \{(G, s, t) : \exists \text{ path } (s \rightarrow t) \in G\} \in \mathbf{NL}$ , 这是平凡的. 事实上它是  $\mathbf{NL}$ -complete 的, 而  $\mathbf{L} = \mathbf{NL}$  是 open 的.

#### 定理 4.1.4 (Space Hierarchy)

若空间可构造的  $f, g$  满足  $f(n) = o(g(n))$ , 则  $\text{SPACE}(f(n)) \subsetneq \text{SPACE}(g(n))$ .

→ Proof. 考虑  $M[\alpha]$ : 模拟运行以  $\mathcal{O}(g(|\alpha|))$  的空间,  $2^{\mathcal{O}(g(|\alpha|))}$  的时间限制模拟  $\mathcal{C}(\alpha)[\alpha]$ , 如果完成运行, 则反转输出; 否则拒绝. 显然  $M \in \text{SPACE}(g(n))$ . 如果  $M \in \text{SPACE}(f(n))$ , 则可以要求  $M \in \text{DTIME}(2^{\mathcal{O}(f(n))})$ , 那么  $M[\mathcal{D}(M)]$  能够完成对  $M$  自己的模拟, 但输出总会导致矛盾.

## § 4.2 空间完全性问题

我们可以仿照时间完全性来定义空间完全性.

#### 定义-命题 4.2.1

考虑 Quantified Boolean Formula (QBF)  $\varphi := Q_1 x_1 \cdots Q_n x_n \phi(x_1, \dots, x_n)$ , 其中  $Q_i \in \{\forall, \exists\}$ ,  $\phi$  是任意 Boolean formula. (可以证明所有量词不在前缀的 QBF 都可以转换成上述形式.)

设  $\text{TQBF} := \{\varphi \in \text{QBF} : \varphi = \text{True}\}$ , 则 TQBF 是  $\mathbf{PSPACE}$ -complete 的.

→ Proof. 显然  $\text{TQBF} \in \mathbf{PSPACE}$  (什么叫做课件写了四页来证, 水课时来了), 下证 TQBF 是  $\mathbf{PSPACE}$ -hard 的. 即说明  $\forall L \in \mathbf{LSPACE}, L \leq_p \text{TQBF}$ .

对给定的  $L$ , 设  $M$  在  $S(n)$  空间判定  $L$ , 而编码  $M$  的格局需要  $m = \mathcal{O}(S(n))$  的空间. 我们需要构造多项式可算的  $f : \{0, 1\}^* \rightarrow \text{QBF}$ , 使得  $f(x) \in \text{TQBF} \Leftrightarrow x \in L$ . 固定输入  $x$ , 对  $M[x]$  的任意两个格局编码  $c, c'$ , 显然存在  $\mathcal{O}(m)$  的 Boolean formula  $\phi(c, c')$  判断转移是否合法. 我们希望检查是否能从  $M[x]$  的初始状态编码  $c_0$  转移到接受状态编码  $c_1$ , 为此, intuitively, 可以倍增地构造

$$\begin{aligned}\psi_0(c, c') &:= \phi(c, c'), \\ \psi_{k+1}(c, c') &:= \exists c'' \psi_k(c, c'') \wedge \psi_k(c'', c').\end{aligned}$$

然而这样的递归展开之后并非多项式长度, 但一个 tricky 的修正:

$$\psi_{k+1}(c, c') := \exists c'' \forall d \forall d' ((d = c \wedge d' = c'') \vee (d = c'' \wedge d' = c')) \rightarrow \psi_k(d, d').$$

就能够让  $\psi_m$  的长度变为  $\mathcal{O}(m^2) = \mathcal{O}(S(n)^2)$ . (其中的  $=$  和  $\rightarrow$  也容易展开为基本逻辑运算.) 事实上这个  $f$  可以  $\mathcal{O}(\log m)$  额外空间地完成计算 (不考虑输入和输出纸带).

直接利用这个归约, 我们知道 TQBF 是  $\mathbf{NPSPACE}$ -hard 的, 所以  $\mathbf{NPSPACE} = \mathbf{PSPACE}$ .

由于 SAT 和  $\neg\text{SAT}$  都是 QBF, 我们知道  $\mathbf{NP}$  和  $\mathbf{coNP}$  都是  $\mathbf{PSPACE}$  的子集.

#### 推论 4.2.2 (Savitch)

对任意  $S = \Omega(\log n)$ ,  $\mathbf{NSPACE}(S(n)) \subset \text{SPACE}(S(n)^2)$ .

→ Proof. 和上面的思路相似. 若  $N \in \mathbf{NSPACE}(S(n))$  判定  $L$ , 对给定输入  $|x| = n$ , 考虑  $N[x]$  的格局图, 它至多存在  $2^m$  个结点 ( $m$  的定义同上), 我们需要判定从初始格局到接受格局的路径的存在性. 同样倍增计算并复用递归的空间即可.

### 推论 4.2.3

TQBF 无法在  $\mathcal{O}(\text{polylog}(n))$  空间内解决.

→ *Proof.* 归约算法是 **L** 的, 进而若 TQBF 在  $\mathcal{O}(\text{polylog}(n))$  空间解决, PSPACE 就不强于这个空间复杂性类, 违背了分层定理.

## § 4.3 小空间算法: NL 与 coNL

大量实践表明, 在不使用复杂提示词工程技术时, 不要使用 GPT ≤ 5.4 来咨询 TCS 问题.

### 定义-命题 4.3.1

$\mathbf{NL} = \mathbf{NSPACE}(\log n)$ .

或者, 称  $M$  是一个对数空间 (log-space) verifier, 当且仅当它是一个 3-tape TM, 满足:

- 输入纸带只读;
- 证书纸带流式只读 (只能从左到右读一次), 整数长度为  $\mathcal{O}(\text{poly}(n))$ ;
- 工作空间不超过  $\mathcal{O}(\log n)$ .

则定义

$\mathbf{NL} = \{L : \exists (M : \text{log-space verifier}), \forall x \in L, \exists u \in \{0, 1\}^{\mathcal{O}(\text{poly}(|x|))}, M(x, u) = 1\}$ .

两种定义等价.

→ *Proof.* 按定义验证即可. 这和 **NP** 两种定义的等价性验证手法完全一致.

为什么一定要流式证书呢?

### 推论 4.3.2

在第二种定义中, 如果将证书强化为只读 (可重复读), 定义出  $\mathbf{NL}^*$ , 则  $\mathbf{NL}^* = \mathbf{NP}$ .

→ *Proof.* 将 **NP** 验证机的运行格局编码作为整数输入给  $\mathbf{NL}^*$  验证机, 这个验证机只需要验证相邻两步转移是否合法, 这的确可以 log-space 完成 (只有指针值需要记入内存).

### 定义 4.3.3

$\mathbf{coNL} = \{L : \neg L \in \mathbf{NL}\}$ .

为了研究 **NL**, **L**, **coNL** 等的关系, 我们同样需要先研究一些完全性问题.

不过, 由于 **P** 强于 **L** 和 **NL**, 我们自然不可能用多项式来在二者中归约. 为此, 我们需要先定义对数空间归约.

### 定义 4.3.4

称  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  隐式对数空间可算 (implicitly log-space computable), 当且仅当

- $f$  的输出长度是多项式的 (它保证了指针可被记录);
- 令  $L_f = \{(x, i) : f(x)_i = 1\}$ ,  $L'_f = \{(x, i) : i \leq |f(x)|\}$ , 则  $L_f \in \mathbf{L}$  且  $L'_f \in \mathbf{L}$ .

称  $B$  可对数空间归约到  $C$ , 记作  $B \leq_1 C$ , 当且仅当存在隐式对数可算的  $f$ , 使得  $\forall x, x \in B \Leftrightarrow f(x) \in C$ .

称  $C$  是 **NL-hard** 的, 当且仅当  $\forall B \in \mathbf{NL}, B \leq_1 C$ ; 若进一步  $C \in \mathbf{NL}$ , 则称  $C$  是 **NL-complete** 的.

由于隐式对数空间可算的函数复合仍保持可算性 (每当外层 TM 需要读取输入时, 调用内层 TM 暴力计算出相应位置的字符), 上述  $\leq_1$  是传递的.

#### 定理 4.3.5

$\text{PATH} = \{(G, s, t) : \exists \text{ path } (s \rightarrow t) \in G\}$  是  $\mathbf{NL}$ -complete 的.

→ *Proof.* 显然  $\text{PATH} \in \mathbf{NL}$ . 对任意  $B \in \mathbf{NL}$ , 设 NDTM  $N$  在  $m = \mathcal{O}(\log n)$  空间判定  $B$ , 尝试构造由它向  $\text{PATH}$  的归约函数  $f$ . 自然地, 可以设计  $f(x) = (G_{N[x]}, C_{\text{start}}, C_{\text{accept}})$ , 其中  $G$  的邻接矩阵每个位置事实上就是转移合法性判断, 所以  $f$  的确是隐式对数空间可算的.

#### 定理 4.3.6 (Immerman-Szelepcsényi)

$\neg \text{PATH} \in \mathbf{NL}$ , 进而  $\mathbf{coNL} = \mathbf{NL}$ .

→ *Proof.* 我们需要设计证明  $G$  中不存在  $s \rightarrow t$  路径的证书. 设  $C_i$  表示  $G$  中不超过  $i$  步从  $s$  可达的点集. 假设验证机已知  $|C_{i-1}|$ , 我们设计证明  $v \notin C_i$  的证书:

- 按照  $C_{i-1}$  中的结点升序列出  $|C_{i-1}|$  个证书, 分别证明每个结点的可达性.

验证机只需要在检查证书合法的同时检查证书提到的每个结点的邻居都不是  $v$  即可.

接下来, 容易据此设计证明  $|C_i| = c$  的证书:

- 升序枚举  $V$  中的结点  $u$ , 分别给出  $u \in C_i$  或者  $u \notin C_i$  的证书.

验证机只需要在检查证书合法的同时计数被证明  $u \in C_i$  的  $u$  的个数, 检查结果是否为  $c$ .

最终, 串连给出这些证书, 最后证明  $t \notin C_{n-1}$  即可.

#### 问题 4.3.7

- 若有向图  $G$  中只有多项式条  $s \rightarrow t$  的路径, 是否能  $\mathbf{NL}$  地估计路径条数? (允许常数误差.)
- 对多大的  $\Delta$ ,  $\text{TQBF} \notin \text{SPACE}(\Delta)$ ?

## § 4.4 多项式层级

$\text{EXACT-INDSET}$  是否属于  $\mathbf{NP}$ ? 我们希望给出同时证明 " $\forall$ " 和 " $\exists$ " 两种量词的证书, 这是困难的. 我们尝试将这些量词加入验证机的定义, 得到 (或许) 比  $\mathbf{NP}$  更广的复杂性类.

#### 定义 4.4.1

复杂性类  $\Sigma_2^{\text{P}}$  包含所有满足如下条件的语言  $L$ :

$$\exists M \in \text{TIME}(\text{poly}(n)), \exists q(n) = \text{poly}(n), \forall x \in \{0, 1\}^*, \\ x \in L \Leftrightarrow \left( \exists u \in \{0, 1\}^{q(|x|)}, \forall v \in \{0, 1\}^{q(|x|)}, M(x, u, v) = 1 \right).$$

同理可以定义  $\Sigma_i^{\text{P}}$ , 即从  $\exists$  出发交替出现  $i$  个量词; 并定义  $\Pi_i^{\text{P}} := \mathbf{co}\Sigma_i^{\text{P}}$ , 也即从  $\forall$  出发交替出现  $i$  个量词.

例如  $\text{EXACT-INDSET} \in \Sigma_2^{\text{P}}$ : 只需要检查  $u$  是一个  $k$ -独立集, 而  $v$  不是一个  $(k+1)$ -独立集即可.

以下性质自然成立:

- $\mathbf{NP} \subset \Sigma_2^{\text{P}}$  (忽略  $v$ ) 和  $\mathbf{coNP} \subset \Sigma_2^{\text{P}}$  (忽略  $u$ ).
- $\mathbf{NP} = \Sigma_1^{\text{P}}, \mathbf{coNP} = \Pi_1^{\text{P}}$ .
- $\Sigma_i^{\text{P}} \subset \Sigma_{i+1}^{\text{P}}, \Pi_i^{\text{P}} \subset \Pi_{i+1}^{\text{P}}$ .

为了让定义更简洁, 我们引入预言机 (oracle TM):

#### 定义 4.4.2

指定任意语言  $O$ , 预言机  $M^O$  是一个具有一条额外特殊纸带 (称为 oracle tape) 和三个特殊状态  $q_{\text{query}}, q_{\text{yes}}, q_{\text{no}}$  的 TM. 当  $M^O$  运行到  $q_{\text{query}}$  状态时, 设 oracle tape 上的字符串为  $s_q$ :

- 若  $s_q \in O$ ,  $M^O$  直接转移到  $s_{\text{yes}}$ ;
- 否则  $M^O$  直接转移到  $s_{\text{no}}$ .

这样, 根据定义, 我们还知道  $\Sigma_{i+1}^P = \text{NP}^{\Pi_i^P}$ ,  $\Pi_{i+1}^P = \text{coNP}^{\Sigma_i^P}$ . 进一步定义出:

#### 定义 4.4.3

$$\text{PH} := \bigcup_{i=1}^{\infty} \Sigma_i^P = \bigcup_{i=1}^{\infty} \Pi_i^P.$$

不过, 这样的 **PH** 是否会坍缩到某个  $\Sigma_k^P$  或者  $\Pi_k^P$  呢? 恭喜你, 你问出了包括  $\text{P} \stackrel{?}{=} \text{NP}$  在内的可数个 open problems!

#### 定理 4.4.4 (多项式层级坍缩)

(a) 若  $\text{P} = \text{NP}$ , 则  $\text{PH} = \text{P}$ .

(b) 对任意  $i \geq 1$ , 若  $\Sigma_i^P = \Pi_i^P$ , 则  $\text{PH} = \Sigma_i^P$ .

→ *Proof.* (a) 归纳地,  $i = 1$  时已有  $\text{P} = \text{NP} = \text{coNP}$ . 对其余  $i$ ,  $\Sigma_i^P = \text{NP}^{\Pi_{i-1}^P} = \text{NP}^{\text{P}} = \text{NP} = \text{P}$ ,  $\Pi_i^P = \text{co}\Sigma_i^P = \text{P}$ . 最终就有  $\text{PH} = \text{P}$ .

(b) 同理.

我们同样也定义这些多项式层级的完全性语言. 例如

$$\Sigma_i^P \text{SAT} := \{\text{satisfiable QBFs of form } \exists u_1 \forall u_2 \cdots Q_i u_i \phi(u_1, \dots, u_i)\}$$

是  $\Sigma_i^P$ -complete 的. 但如果存在 **PH**-complete 的语言, 也明显会导致 **PH** 坍缩.

此外, 由于  $\Sigma_i^P \text{SAT} \leq_p \text{TQBF}$ , 可知  $\text{PH} \subset \text{PSPACE}$ . 但是否取等仍然 open.

## 第五章 布尔电路

### § 5.1 基本定义

#### 定义 5.1.1

一个  $n$  输入 1 输出的 Bool 电路是一个含有  $n$  个源, 1 个汇的 DAG. 所有非源汇结点是标记为  $\wedge, \vee$  或  $\neg$  的门.

对于  $\wedge$  和  $\vee$ , fan-in 为 2; 对于  $\neg$ , fan-in 为 1; 允许 fan-out 大于 1.

一个直接的问题是, Bool 电路的输入规模是固定的, 它无法向 TM 一样用完全相同的内部定义处理任意长度的输入. 为此, 我们一般研究 Bool 电路族, 即对每个输入长度  $n$ , 我们可以按照某种规则对它单独设计电路.

#### 定义 5.1.2

对  $T: \mathbb{N} \rightarrow \mathbb{N}$ , 一个  $T(n)$  规模的电路族是序列  $\{C_n: n \in \mathbb{N}\}$ , 其中  $C_n$  是  $n$  输入 1 输出的, 且  $|C_n| \leq T(n)$ .

定义语言  $L \in \text{SIZE}(T(n))$ , 当且仅当存在  $T(n)$  规模的电路族, 使得  $\forall n, \forall x \in \{0, 1\}^n, x \in L \Leftrightarrow C_n(x) = 1$ .

电路族本身是很强的: 任意函数  $f: \{0, 1\}^n \rightarrow \{0, 1\}$  显然都能被  $n2^n$  规模的电路计算. 我们应当关注那些规模受到约束的电路族.

### 定义 5.1.3

复杂性类  $\mathbf{P}/\text{poly} := \bigcup_{c \geq 0} \text{SIZE}(n^c)$ .

### 定理 5.1.4

$\mathbf{P} \subsetneq \mathbf{P}/\text{poly}$ .

→ *Proof.* 对于包含关系, 容易用常数层电路计算一次格局转移, 那么  $T(n)$  时间的 TM 可被  $T(n)^2$  规模的电路模拟, 且模拟电路深度也是  $\mathcal{O}(T(n))$ .

对于真包含, 考虑  $L \subset \{1^n : n \in \mathbb{N}\} \subset \{0, 1\}^*$ , 则显然  $L \subset \mathbf{P}/\text{poly}$ , 即所有单字符语言都属于  $\mathbf{P}/\text{poly}$ , 随手便能取出一个不可判定语言:

$$\text{UHALT} := \{1^n : n \text{ encodes } (M, x) \text{ where } M[x] \rightarrow H\}.$$

一个有意思的观察是, 证明包含关系电路本身也能够被 TM 在多项式时间内计算出来. 更广泛地:

### 定义 5.1.5

称一个电路族是  $\mathbf{P}$ -uniform 的, 当且仅当存在 TM  $M$ ,  $M[n]$  在多项式时间输出  $C_n$ .

刚刚提到 Bool 函数总能被  $n2^n$  规模的电路计算, 接下来我们说明这个界几乎是紧的:

### 定理 5.1.6

存在  $f: \{0, 1\}^n \rightarrow \{0, 1\}$  (事实上, 对于绝大多数  $f$ ), 它需要至少  $\Omega(2^n/n)$  规模的电路来计算.

→ *Proof.*  $|\{f: \{0, 1\}^n \rightarrow \{0, 1\}\}| = 2^{2^n}$ , 而  $T$  规模电路数量小于有  $\left(3\binom{T}{2}\right)^T \sim (3T^2)^T$ . 则

$$\left(3\binom{T}{2}\right)^T \geq 2^{2^n} \Rightarrow T = \Omega(2^n/n).$$

在  $\mathbf{PSPACE}$  中, 一个典型的 hard 函数是 TQBF.

### 定理 5.1.7

语言  $L \in \mathbf{P}$  当且仅当  $L$  是  $\mathbf{L}$ -uniform 的.

→ *Proof.* 右推左显然 (回忆  $\mathbf{L} \subset \mathbf{P}$ ). 左推右, 设多项式时间的  $M$  判定  $L$ , 沿用  $\mathbf{P} \subset \mathbf{P}/\text{poly}$  的证明过程, 翻译  $M$  的格局转移, 就能得到 log-space 生成判定电路的 TM.

## § 5.2 电路复杂性

### 定义 5.2.1

复杂性类  $\text{DTIME}(T(n))/a(n)$  包含所有满足如下条件的  $L$ :

$$\exists \{\alpha_n\}_{n \geq 1}, \exists M, \forall n, \left( \alpha_n \in \{0, 1\}^{a(n)} \wedge \forall x \in \{0, 1\}^n, M(x, \alpha_n) = 1 \Leftrightarrow x \in L \right).$$

其中  $M$  是  $\mathcal{O}(T(n))$  时间的 TM.

其中 advice  $\alpha_n$  相当于给 TM 外接了一个由  $\alpha_n$  编码的 "电路板" (这个电路本身或许不能在相应复杂度内算出来, 甚至是不可计算的), 以加强 TM 的能力.

注意, 多项式时间的 NDTM 不强这种带 advice 的 TM. 如果尝试用 NDTM 枚举 advice, 我们无法保证对同样长度的不同输入  $x, y$ , NDTM 在同一个 advice 分支下接受  $x, y$ .

### 定理 5.2.2

$$\mathbf{P}/\text{poly} = \bigcup_{c,d \geq 0} \text{DTIME}(n^c)/n^d.$$

→ Proof. 右含于左: 若  $L \in \mathbf{P}/\text{poly}$ , 取判定它的多项式规模电路族  $\{C_n\}$ . 把  $C_n$  的编码作为 advice  $\alpha_n$ , 则一台普通 TM 读入  $(x, \alpha_{|x|})$  后直接模拟电路即可. 因为  $C_n$  的大小是多项式, advice 长度和模拟时间也都是多项式.

左含于右: 若  $L \in \text{DTIME}(n^c)/n^d$ , 则存在 TM  $M$  和 advice  $\alpha_n \in \{0, 1\}^{n^d}$  判定  $L$ . 对每个固定的  $n$ , 我们把  $\alpha_n$  硬编码进模拟  $M$  的电路中; 根据  $\mathbf{P} \subset \mathbf{P}/\text{poly}$  的格局展开证明,  $M$  的  $\mathcal{O}(n^c)$  步计算可以被多项式规模电路模拟. 因而得到判定  $L$  的多项式规模电路族.

### 定理 5.2.3 (non-uniform 规模分层)

若  $n < T(n) < T'(n) < 2^n/(100n)$ , 且  $T(n) \log^2 T(n) = o(T'(n))$ , 则  $\text{SIZE}(T(n)) \subsetneq \text{SIZE}(T'(n))$ .

→ Proof. 设  $\ell = \log T(n) + \log \log T(n) + C$ , 其中常数  $C$  足够大. 则 counting argument 可以保证存在  $f: \{0, 1\}^\ell \rightarrow \{0, 1\}$ , 它要求至少  $\Omega(2^\ell/\ell) > T(n)$  规模的电路计算, 所以  $f \notin \text{SIZE}(T(n))$ .

### 定理 5.2.4 (Karp-Lipton)

若  $\mathbf{NP} \subset \mathbf{P}/\text{poly}$ , 则  $\mathbf{PH} = \Sigma_2^{\mathbf{P}}$ .

→ Proof. 我们的思路是

$$\mathbf{NP} \subset \mathbf{P}/\text{poly} \Rightarrow \Pi_2\text{SAT} \in \Sigma_2^{\mathbf{P}} \Rightarrow \Pi_2^{\mathbf{P}} \subset \Sigma_2^{\mathbf{P}} \Rightarrow \mathbf{PH} = \Sigma_2^{\mathbf{P}}.$$

具体来讲, 我们需要在 given  $\mathbf{NP} \subset \mathbf{P}/\text{poly}$  的情况下, 在多项式时间调换  $\Pi_2\text{SAT}$  的 " $\forall \cdot \exists$ ." 量词顺序. 设语言

$$L := \{(\phi, u) : \exists v \phi(u, v) = 1\} \in \mathbf{NP} \stackrel{\text{conjecture}}{\subset} \mathbf{P}/\text{poly},$$

存在多项式规模电路族  $\{C_n\}$  判定  $L$ , 即  $\forall(\phi, u), (\exists v, \phi(u, v) = 1) \Leftrightarrow C_n(\phi, u) = 1$ . 利用 search argument (依次猜测  $v$  的每个 bit, 检查合法性), 我们可以得到多项式规模电路族  $\{C'_n\}$ , 它直接输出  $v = C'_n(\phi, u)$ , 对  $C'_n$  的描述  $w$ , 作为一个多项式的字符串, 总是存在.

因而,  $\forall u \exists v \phi(u, v) = 1 \Leftrightarrow \exists w \forall u \phi(u, C'_n(\phi, u)) = 1$ . 明所欲证.

## § 5.3 浅层电路

除开电路规模, 另一个重要的指标自然是电路的层数: 它衡量了并行算法的效率.

### 定义 5.3.1

称函数  $f: \{0, 1\}^* \rightarrow \{0, 1\}$  是可高效并行计算的, 当且仅当对固定输入长度  $n$ , 存在使用  $\text{poly}(n)$  个处理器的并行算法在  $\mathcal{O}(\text{polylog}(n))$  计算  $f$ .

### 定义 5.3.2



复杂性类  $\mathbf{NC}^i$  包含所有能够被规模为  $\mathcal{O}(\text{poly}(n))$  的, log-space uniform 的, 深度为  $\mathcal{O}(\log^i n)$  的电路族判定的语言. 复杂性类  $\mathbf{AC}^i$  在此基础上允许  $\wedge$  和  $\vee$  门拥有任意数量的 fan-in.

显然  $\mathbf{NC}^i \subset \mathbf{AC}^i \subset \mathbf{NC}^{i+1}$ . 特别地,  $\mathbf{NC}^0 \subsetneq \mathbf{AC}^0 \subsetneq \mathbf{NC}^1$ . 对于第一个不等号,  $\mathbf{NC}^0$  显然无法计算  $n$  个 bit 的与. 对于第二个不等号, 经典的例子是  $n$ -bit xor/majority/threshold (to-be proved). 对其他的  $i$ , 真包含与否仍然 open.

此外, 容易看出,  $f$  是可高效并行计算的, 当且仅当  $f \in \mathbf{NC}^*$  (这一点就要求电路是 uniform 的, 不然 TM 无法使用).

### 定理 5.3.3

$\text{XOR}_n \notin \mathbf{AC}^0$ , 这进一步给出  $\mathbf{AC}^0 \subsetneq \mathbf{NC}^1$ .

→ Proof. 使用 Switching Lemma 和一些炫酷的随机分析技术证明. 这里摆了 (悲).

## 第六章 概率图灵机

### § 6.1 PTM 的定义与基本性质

#### 定义 6.1.1

概率图灵机 (PTM)  $M$  具有两个转移函数  $\delta_0, \delta_1$ , 在每次转移时, 独立以  $\frac{1}{2}$  的概率在二者中选择一个进行.

#### 定义 6.1.2

对  $T: \mathbb{N} \rightarrow \mathbb{N}$  和语言  $L$ , 称 PTM  $M$  在  $T(n)$  判定  $L$ , 当且仅当对任意  $x$ ,  $M$  在  $T(|x|)$  内停机, 且  $\Pr[M_r(x) = [x \in L]] \geq 2/3$ .

双边错误类  $\mathbf{BPTIME}(T(n))$  为所有能被 PTM 在  $\mathcal{O}(T(n))$  时间判定的语言.

$\mathbf{BPP} := \bigcup_{c \geq 1} \mathbf{BPTIME}(n^c)$ ; 或者, 等价地,  $L \in \mathbf{BPP}$  当且仅当

$$\exists \text{PTM } M, \exists p(n) = \text{poly}(n), \forall x \in \{0, 1\}^*,$$

$$\left( x \in L \Rightarrow \Pr_{r \sim \mathcal{U}_p(|x|)} [M_r(x) = 1] \geq 2/3 \right) \wedge \left( x \notin L \Rightarrow \Pr_{r \sim \mathcal{U}_p(|x|)} [M_r(x) = 1] < 1/3 \right).$$

单边错误类  $\mathbf{RTIME}(T(n))$  在双边错误的基础上要求  $x \notin L \Rightarrow \Pr[M_r(x) = 0]$ ,  
 $\mathbf{RP} := \bigcup_{c \geq 1} \mathbf{RTIME}(n^c)$ .

无错误类  $\mathbf{ZTIME}(T(n))$  要求 PTM 回答总是正确, 但只要求期望停机时间为  $\mathcal{O}(T(n))$ ,  
 $\mathbf{ZPP} = \bigcup_{c \geq 1} \mathbf{ZTIME}(n^c)$ .

可见:

- $\mathbf{BPP} = \mathbf{coBPP}$ .
- $\mathbf{RP} \subset \mathbf{BPP}$ .
- $\mathbf{coRP} \subset \mathbf{BPP}$ .

#### 定理 6.1.3

$\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP}$ .

→ Proof. 左含于右: 给定  $A \in \mathbf{ZPP}$  的 PTM  $M$ , 设定足够大的运行时间上限  $p^*(n)$ , 模拟运行  $M$ , 当步数超过  $p^*(n)$  时直接回答. 右含于左: 反复运行两个单边错误的 PTM, 直到有一者给出确定性答案.

#### 定理 6.1.4 (error reduction)



若存在多项式时间的 PTM  $M$  和常数  $c$ , 使得对语言  $L$  有  
 $\forall x \in \{0, 1\}^*, \Pr[M(x) = L(x)] \geq 0.5 + |x|^{-c}$ , 那么存在常数  $d$  和另一多项式时间的 PTM  $M'$  使得  $\forall x \in \{0, 1\}^*, \Pr[M'(x) = L(x)] \geq 1 - 2^{-|x|^d}$ .

→ *Proof.* 记原机器单次正确概率为  $\frac{1}{2} + \gamma$ , 其中  $\gamma \geq n^{-c}$ . 令  $M'$  独立运行  $M$  共  $m$  次并取多数答案. 若  $X_i$  表示第  $i$  次是否正确, 则  $\mathbb{E}X_i \geq \frac{1}{2} + \gamma$ , 于是 Chernoff/Hoeffding 给出

$$\Pr \left[ \sum_{i=1}^m X_i \leq m/2 \right] \leq \exp(-2m\gamma^2).$$

取  $m = Cn^{2c+d}$  (常数  $C$  足够大), 就有失败概率至多  $2^{-n^d}$ . 由于  $m$  仍然是多项式,  $M'$  仍是多项式时间 PTM. 小规模输入可以直接调整常数或硬编码特判, 不影响复杂性类.

### 定理 6.1.5

**BPP**  $\subset$  **P**/poly.

→ *Proof.* 通过 error reduction, 可以取失败概率不超过  $2^{-(n+1)}$  的 PTM  $M$ . 它在所有  $\{0, 1\}^n$  上一定存在一个全部正确的随机串, 我们直接将这个串作为 advice 即可.

### 定理 6.1.6

**BPP**  $\subset \Sigma_2^P \cap \Pi_2^P$ .

→ *Proof.* 对 **BPP** 的判定机  $M$ , 仍然先用 error reduction, 它使用  $m = \text{poly}(n)$  个 bit, 但保证错误率不超过  $2^{-n}$ . 现在令  $k = \lceil m/n \rceil + 1$ , 考虑:

$$\exists u_1, \dots, u_k \in \{0, 1\}^m, \forall r \in \{0, 1\}^m, \bigvee_{i=1}^k M(x, r \oplus u_i).$$

讨论:

- 若  $x \notin L$ , 那么  $\Pr_r[M(x, r \oplus u) = 1] \leq 2^{-n}$  总是成立, union bound 给出

$$\Pr_r \left[ \bigvee_{i=1}^k M(x, r \oplus u_i) = 1 \right] \leq k2^{-n} < 1.$$

因此必然存在一个  $r$  让表达式为 0.

- 若  $x \in L$ , 那么  $\Pr_u[M(x, r \oplus u) = 0] \leq 2^{-n}$  也总是成立, 那么

$$\Pr_{u_1, \dots, u_k} \left[ \bigvee_{i=1}^k M(x, u_i \oplus r) = 0 \right] \leq 2^{-kn} < 2^{-m},$$

同样 union bound 告诉我们

$$\Pr_{u_1, \dots, u_k} \left[ \exists r, \bigvee_{i=1}^k M(x, u_i \oplus r) = 0 \right] < 1.$$

因此一定存在一列  $u_1, \dots, u_k$  满足条件.

到此 **BPP**  $\in \Sigma_2^P$ , 再结合 **coBPP** = **BPP** 就得到目标.

*Remark.*

这个随机平移覆盖的技术非常重要, 从形式上它提供了将  $\Pr[\dots] \geq 1 - \varepsilon$  变为一个  $\Sigma_2^P$  SAT 确定命题  $\exists \dots \forall \dots \bigvee \dots = 1$ , 因而在后续关于交互式证明的讨论这种我们也会看到这种技术.

### 定义 6.1.7

语言  $L \in \mathbf{BPL}$  当且仅当存在  $\mathcal{O}(\log n)$  空间, 多项式时间 (因为奇怪的原因人们省略了  $\mathbf{BPLP}$  最后的 "P") 的 PTM  $M$  使得  $\Pr[M(x) = L(x)] \geq 2/3$ ;

$\mathbf{RL}$  在  $\mathbf{BPL}$  的基础上要求  $\forall x \notin L, \Pr[M(x) = 1] = 0$ .

自然地有  $\mathbf{RL} \subset \mathbf{NL} \subset \mathbf{P}$ . 此外也有  $\mathbf{BPL} \subset \mathbf{P}$ : 展开  $\mathbf{BPL}$  机器对应的多项式规模的 Markov 链, 多项式地 Gauss 消元求出游走到游走到接受格局的概率即可. 我们还能进一步证明  $\mathbf{BPL} \subset \mathbf{L}^2$  (分治矩阵乘法, 不加任何记忆化地暴力递归; 双对数来源于浮点精度保持和递归状态记录). 通过对转移矩阵的进一步性质挖掘, 事实上可以获得  $\mathbf{BPL} \subset \mathbf{L}^{3/2}$  的界.

## § 6.2 概率复杂性类小结

把上面的结论整理一下, 可以得到如下关系图:

$$\mathbf{P} \subset \mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP} \subset \mathbf{RP} \cup \mathbf{coRP} \subset \mathbf{BPP},$$

并且

$$\mathbf{BPP} = \mathbf{coBPP}, \quad \mathbf{BPP} \subset \mathbf{P}/\text{poly}, \quad \mathbf{BPP} \subset \Sigma_2^{\mathbf{P}} \cap \Pi_2^{\mathbf{P}}.$$

这些包含多数都不是因为随机性本身很弱, 而是因为我们把错误率压到极小, 再用 union bound 或随机平移覆盖把随机命题改写成确定性命题. 这也是概率算法里最常见的"先放大, 再固定随机串"套路.

### 命题 6.2.1

若  $\mathbf{BPP} = \mathbf{EXP}$ , 则  $\mathbf{EXP} \subset \mathbf{P}/\text{poly}$ , 从而与一般的电路下界直觉严重冲突. 因此人们普遍相信  $\mathbf{BPP}$  更接近  $\mathbf{P}$  而不是  $\mathbf{EXP}$ .

这当然不是证明, 只是一个复杂性世界观提示: 随机性看起来很有用, 但在许多理论语境下, 它可能只是被短种子伪随机地模拟了.

## 第七章 交互式证明

### § 7.1 IP 的定义与基本性质

在  $\mathbf{NP}$  的证书定义中, prover 一次性把证明写在纸上, verifier 只负责检查. 交互式证明则允许 verifier 反复提问, prover 反复回答; verifier 的随机性是关键, 没有它时事情会退回  $\mathbf{NP}$ .

#### 定义 7.1.1 (DIP)

称语言  $L$  有  $k$  轮确定性交互式证明, 当且仅当存在多项式时间确定性 TM  $V$ , 它在输入  $x$  和历史消息后生成下一条问题, 并和任意函数  $P: \{0, 1\}^* \rightarrow \{0, 1\}^*$  交互  $k$  轮, 满足

$$\begin{aligned} x \in L &\Rightarrow \exists P, \text{Out}_V(V, P)(x) = 1, \\ x \notin L &\Rightarrow \forall P, \text{Out}_V(V, P)(x) = 0. \end{aligned}$$

$\mathbf{DIP}$  表示所有具有多项式轮数确定性交互式证明的语言.

#### 定理 7.1.2

$$\mathbf{DIP} = \mathbf{NP}.$$

→ Proof.  $\mathbf{NP} \subset \mathbf{DIP}$  显然: prover 在第一轮直接发送  $\mathbf{NP}$  证书即可.

反过来, 若  $L \in \mathbf{DIP}$ , 则对  $x \in L$  存在一个让  $V$  接受的交互 transcript. 由于  $V$  是确定性的,  $\mathbf{NP}$  verifier 只需把整段 transcript 当证书, 逐轮检查  $V$  的问题是否和 transcript 一致, 最后检查  $V$  是否接受. transcript 长度是多项式, 因而  $L \in \mathbf{NP}$ .

### 定义 7.1.3 (IP)

称  $L \in \mathbf{IP}[k]$ , 当且仅当存在多项式时间 PTM  $V$ , 它可以和任意函数  $P : \{0, 1\}^* \rightarrow \{0, 1\}^*$  交互  $k$  轮, 且

$$\begin{aligned} x \in L &\Rightarrow \exists P, \Pr_r[\text{Out}_V\langle V, P \rangle(x) = 1] \geq 2/3, \\ x \notin L &\Rightarrow \forall P, \Pr_r[\text{Out}_V\langle V, P \rangle(x) = 1] \leq 1/3. \end{aligned}$$

概率只来自 verifier 的私有随机串  $r$ . 定义

$$\mathbf{IP} := \bigcup_{c \geq 0} \mathbf{IP}[n^c].$$

上面的两个条件照例叫 completeness 和 soundness. 注意 prover 没有计算能力限制, 所以随机化 prover 不会增强模型: 对  $x \in L$ , 随机 prover 的平均成功率若高, 就存在某个随机性固定后的确定性 prover 也高; 对  $x \notin L$ , 任意随机 prover 都只是确定性 prover 的分布, soundness 由平均保持.

### 命题 7.1.4 (误差放大)

在  $\mathbf{IP}$  的定义中, 将  $2/3$  替换为  $1 - 2^{-n^c}$ , 将  $1/3$  替换为  $2^{-n^c}$ , 不改变  $\mathbf{IP}$ .

→ *Proof.* verifier 用独立随机串并行/顺序重复协议  $m = \text{poly}(n)$  次, 最后多数投票. 对  $x \in L$ , 使用同一个诚实 prover 即可; 对  $x \notin L$ , 任意 prover 在每次实验中诱导出的接受概率都被  $1/3$  控制. Chernoff bound 直接给出  $2^{-\Omega(m)}$  的错误率.

但把 completeness 直接替换成 1 是非平凡的; 把 soundness 替换成 0 则会让 verifier 本质上失去随机检查的能力, 模型退回确定性证明.

### 定理 7.1.5

$$\mathbf{IP} \subset \mathbf{PSPACE}.$$

→ *Proof.* 固定  $V$  和输入  $x$ , 考虑所有可能的交互历史. PSPACE 算法递归计算"从当前历史开始, 最优 prover 能让 verifier 最终接受的最大概率". 在 verifier 轮, 对所有随机选择取平均; 在 prover 轮, 对所有可能回答取最大值. 递归深度和每条消息长度都是多项式, 每层只需复用空间并维护若干计数器, 因而总空间是多项式. 最后比较最优接受概率是否至少  $2/3$  即可.

### 例子 7.1.6 (Graph Non-Isomorphism)

设

$$\text{GNI} := \{(G_0, G_1) : G_0 \not\cong G_1\}.$$

它有一个非常短的交互式证明:

- $V$  随机取  $i \in \{0, 1\}$  和随机置换  $\pi$ , 发送  $H = \pi(G_i)$  给  $P$ ;
- $P$  回答  $j$ , 表示它认为  $H$  来自  $G_j$ ;
- $V$  接受当且仅当  $i = j$ .

若  $G_0 \not\cong G_1$ , 无界 prover 能完全识别  $H$  来自哪一边, completeness 为 1. 若  $G_0 \cong G_1$ , 那么  $H$  的分布与  $i$  无关, 任意 prover 猜中  $i$  的概率至多  $1/2$ . 重复常数次即可把 soundness 压到  $1/3$ .

## § 7.2 公共随机与 AM

在一般 IP 中, verifier 的随机串是私有的. 若要求 verifier 发出的消息只是不加隐藏的随机串, 就得到 public coin proof, 也称 Arthur-Merlin proof.

### 定义 7.2.1 (AM)

$\mathbf{AM}[k]$  是  $\mathbf{IP}[k]$  的子类, 额外要求 verifier 的每条消息都是新鲜随机串, 且 verifier 不能使用其他私有随机性.

有一些基本事实:

- $\mathbf{AM}[2] = \mathbf{BP} \cdot \mathbf{NP}$ , 即先随机选择 Arthur 消息, 再让 Merlin 给出 NP 型证书;
- 对任意常数  $k \geq 2$ ,  $\mathbf{AM}[k] = \mathbf{AM}[2]$ ;
- 一般私有随机协议也能转化为公共随机协议:  $\mathbf{IP}[k] \subset \mathbf{AM}[k+2]$ .

下面这个 set lower bound protocol 是公共随机化的核心小工具.

### 引理 7.2.2 (集合下界协议)

设  $S \subset \{0, 1\}^m$ , 且  $x \in S$  可以由多项式证书验证. 双方知道整数  $K$ , 并取  $k$  使得  $2^{k-1} < K \leq 2^k$ . 存在一个公共随机协议用于区分:

$$|S| \geq K \quad \text{v.s.} \quad |S| \leq K/2.$$

→ *Proof.* 取 pairwise independent hash family  $\mathcal{H}_{m,k}$ , 其中  $h: \{0, 1\}^m \rightarrow \{0, 1\}^k$ . verifier 随机取  $h \in \mathcal{H}_{m,k}$  和  $y \in \{0, 1\}^k$ , 发给 prover. prover 试图给出  $x \in S$  及其证书, 满足  $h(x) = y$ . verifier 验证二者.

若  $|S| \leq K/2$ , 对任意  $h$ , 随机  $y$  命中  $h(S)$  的概率至多  $|S|/2^k \leq 1/4$ . 若  $|S| \geq K$ , 则取任意  $S' \subset S$  且  $|S'| = K$ , pairwise independence 和 inclusion-exclusion/Chebyshev 型估计说明随机  $h, y$  命中  $h(S')$  的概率至少为常数 (可取约  $3/8$ ), 因而命中  $h(S)$  的概率也至少为常数. 通过重复即可放大间隔.

于是, GNI 也可以写成  $\mathbf{AM}$  协议. 对图  $G_0, G_1$ , 令

$$S = \{(H, \pi) : (H \cong G_0 \vee H \cong G_1) \wedge \pi \in \text{Aut}(H)\}.$$

其中  $H \cong G_i$  的证书是一组置换,  $\pi \in \text{Aut}(H)$  可直接检查. 如果  $G_0 \not\cong G_1$ , 则  $|S| = 2n!$ ; 如果  $G_0 \cong G_1$ , 则  $|S| = n!$ . 这正好是 set lower bound protocol 能分辨的大小差距.

## § 7.3 IP 与 PSPACE

结合 定理 7.1.5, 只需证明  $\mathbf{PSPACE} \subset \mathbf{IP}$ . 由于 TQBF 是  $\mathbf{PSPACE}$ -complete 的, 我们只需要给 TQBF 设计交互式证明.

第一步是算术化 (arithmetization): 把 Boolean 公式翻译为域  $\mathbb{F}_p$  上的多项式. 例如

$$\neg z \mapsto 1 - z, \quad z \wedge y \mapsto zy, \quad z \vee y \mapsto 1 - (1 - z)(1 - y).$$

这样, 对 Boolean 输入  $b \in \{0, 1\}^n$ , 多项式  $P_\phi(b)$  和原公式  $\phi(b)$  一致.

### 引理 7.3.1 (Sumcheck)

给定低次数多项式  $g(x_1, \dots, x_n)$ , 素数  $p$ , 和声称值  $k$ , 存在交互式证明验证

$$k = \sum_{b_1 \in \{0,1\}} \cdots \sum_{b_n \in \{0,1\}} g(b_1, \dots, b_n) \pmod{p}.$$

→ *Proof.* verifier 自己无法枚举  $2^n$  项, 所以让 prover 逐轮发送一元多项式. 第一轮 prover 发送

$$s_1(x_1) \stackrel{?}{=} \sum_{b_2, \dots, b_n \in \{0,1\}} g(x_1, b_2, \dots, b_n).$$

verifier 检查  $s_1(0) + s_1(1) = k$ . 若通过, 随机取  $a_1 \in \mathbb{F}_p$ , 递归检查

$$s_1(a_1) = \sum_{b_2, \dots, b_n \in \{0,1\}} g(a_1, b_2, \dots, b_n).$$

最后只剩零个求和变量时, verifier 直接计算  $g(a_1, \dots, a_n)$ .

completeness 显然为 1. soundness 来自 Schwartz-Zippel: 若 prover 发送的  $s_i$  不是正确多项式, 它和正确多项式在随机点上相等的概率至多  $\deg / p$ . 因而整体欺骗概率至多约  $dn/p$ , 取足够大的  $p$  即可压到  $1/3$  以下.

对 #SAT, 直接把 # $\phi$  写成  $\sum_b P_\phi(b)$  就能使用 sumcheck. 对 TQBF, 量词可以看成多项式算子:  $\exists x_i$  对应  $\sum_{x_i \in \{0,1\}}$ ,  $\forall x_i$  对应  $\prod_{x_i \in \{0,1\}}$  (用非零性表达真值). 形式上, 例如

$$\forall x_1 \exists x_2 \dots \exists x_n \phi \rightsquigarrow \prod_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_n \in \{0,1\}} P_\phi(b_1, \dots, b_n) \neq 0.$$

问题是乘法会让次数膨胀. 课件中的修正是在线性化算子

$$L_i(p)(x_1, \dots, x_n) := x_i p(x_1, \dots, 1, \dots, x_n) + (1 - x_i) p(x_1, \dots, 0, \dots, x_n)$$

的帮助下, 在每轮操作后把当前多项式重新变成关于相关变量的多线性多项式. 因为当  $x_i \in \{0, 1\}$  时  $L_i(p)$  与  $p$  一致, 它不改变原 QBF 在 Boolean cube 上的值, 但保证中间多项式保持低次数表示.

### 定理 7.3.2

**IP = PSPACE.**

→ Proof. **IP**  $\subset$  **PSPACE** 已证. 对反向包含, 将 TQBF arithmetize, 并把交替量词与线性化算子组成一个长度多项式的算子序列:

$$Q = Q_1 L_1 Q_2 L_1 L_2 \dots Q_n L_1 \dots L_n P_\phi.$$

verifier 从外向内检查 prover 声称的当前多项式值. 若当前算子是求和/乘积, 就检查边界等式并随机固定一个变量; 若当前算子是  $L_i$ , 就检查对应的线性化等式并继续随机递归. 由于每一步只处理低次数一元多项式, prover 的消息长度始终是多项式. 正确命题有诚实 prover 可使 verifier 总接受; 错误命题若想骗过 verifier, 必须在某轮让两个不同低次数多项式在随机点碰巧相等, 概率由 Schwartz-Zippel 控制. 因此  $\text{TQBF} \in \text{IP}$ , 也就有  $\text{PSPACE} \subset \text{IP}$ .

## 第八章 量子计算理论初步\*

### § 8.1 量子态与操作

#### 定义 8.1.1 (Qubit)

一个 qubit 的纯态是

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle, \quad \alpha_0, \alpha_1 \in \mathbb{C}, \quad |\alpha_0|^2 + |\alpha_1|^2 = 1.$$

$n$  个 qubit 的纯态是

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1.$$

复合系统的状态空间是张量积. 例如  $|\psi\rangle$  和  $|\varphi\rangle$  的 joint state 是  $|\psi\rangle \otimes |\varphi\rangle$ . 不是所有  $n$ -qubit 态都能拆成单 qubit 态的张量积, 这就是纠缠态出现的地方.

#### 定义 8.1.2 (Mixed State)

若系统以概率  $p_i$  处于纯态  $|\psi_i\rangle$ , 则它的 density operator 是

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|.$$

### 定义 8.1.3 (Quantum Operation)

闭合量子系统中的计算步骤由 unitary matrix  $U$  描述, 即

$$U^\dagger U = I.$$

它保持范数, 因而保持总概率. 常见单比特门包括

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

多 qubit 门通过张量积和作用在部分寄存器上的 unitary 给出.

### 定义 8.1.4 (Measurement)

测量由一组算子  $\{M_m\}$  描述, 满足 completeness equation

$$\sum_m M_m^\dagger M_m = I.$$

若测量前状态为  $|\psi\rangle$ , 则结果  $m$  出现的概率为

$$p(m) = \langle\psi|M_m^\dagger M_m|\psi\rangle,$$

测量后的状态为

$$\frac{M_m|\psi\rangle}{\sqrt{p(m)}}.$$

最常用的是计算基测量:  $M_x = |x\rangle\langle x|$ . 若  $|\psi\rangle = \sum_x \alpha_x |x\rangle$ , 则测得  $x$  的概率为  $|\alpha_x|^2$ .

## § 8.2 量子计算模型与 BQP

### 定义 8.2.1

称 Boolean 函数  $f: \{0, 1\}^* \rightarrow \{0, 1\}$  可在 quantum  $T(n)$ -time 内计算, 当且仅当存在一台经典多项式时间 TM, 输入  $(1^n, 1^{T(n)})$  时输出量子门序列  $F_1, \dots, F_T$ , 且对任意  $x \in \{0, 1\}^n$ :

- 初始化  $m \leq T(n)$  个 qubit 为  $|x0\dots 0\rangle$ ;
- 依次作用  $F_1, \dots, F_T$ ;
- 在计算基下测量寄存器, 输出第一位;
- 输出等于  $f(x)$  的概率至少为  $2/3$ .

### 定义 8.2.2

**BQP** 是所有可被 quantum  $\text{poly}(n)$ -time 计算的 Boolean 语言.

和 **BPP** 一样, **BQP** 的常数错误率可以用独立重复和多数投票放大. 显然  $\mathbf{P} \subset \mathbf{BPP} \subset \mathbf{BQP}$ : 量子电路可以用 Hadamard 门产生均匀随机 bit, 然后可逆地模拟 classical randomized computation.

### 例子 8.2.3 (Grover Search)

设  $f: \{0, 1\}^n \rightarrow \{0, 1\}$  恰有一个满足  $f(w) = 1$  的  $w$ . 经典随机查询需要  $\Theta(2^n)$  次才能稳定找到  $w$ , 而 Grover 算法只需  $\mathcal{O}(2^{n/2})$  次查询.

粗略地说, 先制备均匀态

$$|u\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \{0,1\}^n} |x\rangle, \quad N = 2^n.$$

每一轮先用 oracle 把目标态  $|w\rangle$  的相位翻转, 再围绕  $|u\rangle$  做反射. 这两个反射的乘积会在  $\text{span}\{|w\rangle, |u\rangle\}$  中逐步旋转, 大约  $\frac{\pi}{4}\sqrt{N}$  轮后, 测量得到  $w$  的概率为常数. 这就是量子算法里最经典的"幅度放大".

## 第九章 伪随机理论\*

### § 9.1 密码学中的伪随机

密码学中的核心观点是: 真随机很贵, 但只要多项式时间算法分辨不出来, 伪随机就足够像真的.

#### 定义 9.1.1 (Perfect Secrecy)

设  $(E, D)$  是消息长  $m$ , 密钥长  $n$  的加密方案. 若对任意消息  $x, x'$  和任意密文  $z$ , 随机密钥下  $E_k(x)$  与  $E_k(x')$  的分布完全相同, 则称它 perfect secret.

one-time pad  $E_k(x) = x \oplus k$  是 perfect secret, 但要求密钥长度至少为消息长度, 且不能复用. 因此现代密码学通常改用计算安全性.

#### 定义 9.1.2 (Computational Security)

称加密方案  $(E, D)$  是计算安全的, 当且仅当对任意概率多项式时间算法  $A$ , 存在 negligible 函数  $\varepsilon(n) = n^{-\omega(1)}$ , 使得  $A$  从密文中预测任意消息位的优势至多  $\varepsilon$ :

$$\Pr_{k,x}[A(E_k(x)) = (i, x_i)] \leq \frac{1}{2} + \varepsilon(n).$$

#### 定义 9.1.3 (OWF)

多项式时间可计算函数  $f$  是 one-way function, 当且仅当对任意概率多项式时间算法  $A$ , 存在 negligible 函数  $\varepsilon$ , 使得

$$\Pr_{x \sim \{0,1\}^n, y=f(x)}[f(A(y)) = y] < \varepsilon(n).$$

OWF 的存在是密码学中的基本假设之一. 若  $\mathbf{P} = \mathbf{NP}$ , 则可用决策到搜索的标准方法反推出任意多项式可计算函数的原像, 因而 OWF 不存在. 所以 OWF 存在蕴含  $\mathbf{P} \neq \mathbf{NP}$ .

#### 定义 9.1.4 (Cryptographic PRG)

函数  $G: \{0,1\}^n \rightarrow \{0,1\}^{\ell(n)}$  是安全 PRG, 当且仅当  $\ell(n) > n$  且对任意概率多项式时间区分器  $A$ , 存在 negligible 函数  $\varepsilon$ , 使得

$$|\Pr[A(G(U_n)) = 1] - \Pr[A(U_{\ell(n)}) = 1]| < \varepsilon(n).$$

#### 定理 9.1.5 (HILL, Goldreich-Levin 方向)

若 OWF 存在, 则对任意常数  $c$ , 存在 stretch  $\ell = n^c$  的安全 PRG.

这里课件只给路线: one-way permutation 配合 hard-core predicate 可先得到一位 stretch 的 PRG; next-bit unpredictability 蕴含 pseudorandomness; 再通过迭代把 stretch 放大到任意多项式长度.



## § 9.2 空间有界计算的伪随机

复杂性理论中的 PRG 更关心"骗过某类算法". 对空间有界机器, Nisan generator 能用  $\mathcal{O}(\log^2 n)$  个真随机 bit 生成多项式长度的伪随机串.

### 定义 9.2.1

随机变量  $X$  的 min entropy 定义为最大的  $k$ , 使得对所有  $x$  都有

$$\Pr[X = x] \leq 2^{-k}.$$

若  $X$  分布在  $\{0, 1\}^n$  上且  $H_\infty(X) \geq k$ , 则称  $X$  是一个  $(n, k)$ -source.

### 定义 9.2.2

同一有限空间  $\Omega$  上随机变量  $X, Y$  的 statistical distance 为

$$\Delta(X, Y) = \frac{1}{2} \sum_{a \in \Omega} |\Pr[X = a] - \Pr[Y = a]|.$$

### 定义 9.2.3 (Extractor)

函数  $\text{Ext} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$  是  $(k, \varepsilon)$  extractor, 当且仅当对任意  $(n, k)$ -source  $X$ ,

$$\Delta(\text{Ext}(X, U_d), U_m) \leq \varepsilon.$$

### 引理 9.2.4 (Leftover Hash Lemma)

令  $m = k - 2 \log(1/\varepsilon)$ . 若  $H$  从 pairwise independent hash family  $\{h : \{0, 1\}^n \rightarrow \{0, 1\}^m\}$  中均匀选取, 则对任意  $(n, k)$ -source  $X$ ,

$$\Delta((H(X), H), (U_m, H)) < \varepsilon.$$

### 定理 9.2.5 (Nisan Generator)

对任意常数  $c$ , 存在  $c' > 0$  和多项式时间可算函数

$$g : \{0, 1\}^{c' \log^2 n} \rightarrow \{0, 1\}^{n^c},$$

使得任意配置图大小不超过  $n^c$  的 read-once randomness tape 空间有界 TM  $M$  都满足

$$\left| \Pr_{r \sim U_{n^c}} [M(x, r) = 1] - \Pr_{z \sim U_{c' \log^2 n}} [M(x, g(z)) = 1] \right| < 1/10.$$

课件给出的构造可递归描述为: 取一族 extractor

$$\text{Ext}_k : \{0, 1\}^{kr} \times \{0, 1\}^r \rightarrow \{0, 1\}^{kr},$$

定义

$$G_k(a \circ z) = \begin{cases} z, & k = 1; \\ G_{k-1}(a) \circ G_{k-1}(\text{Ext}_{k-1}(a, z)), & k > 1. \end{cases}$$

直观上,  $a$  负责生成前半段伪随机串, 而  $\text{Ext}(a, z)$  从已有高熵信息和少量新随机性中抽取出看起来新鲜的种子来生成后半段. 这就是"把短随机种子递归掰成很长的伪随机 tape".

### 推论 9.2.6

$$\text{BPL} \subset \text{SPACE}(\log^2 n).$$

→ *Proof.* **BPL** 机器的空间是  $\mathcal{O}(\log n)$ , 多项式时间运行的配置图大小也是多项式. 用 Nisan generator 将它所需的多项式长度随机串替换成  $\mathcal{O}(\log^2 n)$  长的种子, 然后确定性地枚举所有种子并模拟, 统计接受比例是否超过阈值即可. 枚举种子和模拟机器共用  $\mathcal{O}(\log^2 n)$  空间.

## 附录: TOC/advice

其实是 cheat sheet 啦, 毕竟加上 advice 会让你的做题能力产生质的飞跃 (笑).

需要 clarify 的是, 课程内容真的很难, 作业也真的真的真的很难, 但考题一般不会有长思考路径的刁难, 基本上就是 apply 一个 advice 然后推一下细节就能搞定. 最重要的是一个准备良好的 advice: (a) 往年题详细过程 (问就是可以直接速通几道大题), (b) complexity zoo, (c) 关键结论的证明技术 (比如我们 26 年某题就要求你知道 TQBF 的 **PSPACE**-hard 性的来源, 才能快速给出证明).

我的 advice 如下. 往年题参考自 22 年和 24 年, 作业题就是 26 年的.  $2 \times 2$  缩印, 其实还有很多空间, 你可以自己加料.

## 知识点

[A1] 非正则语言类对  $\cap$  不封闭. **Prf.** 两个非正则的交可以为空.

[A2] 正则语言  $A$  的 reverse  $A^R$  也正则. **Prf.** 用 NFA 模拟反向 DFA.

[A3 Myhill-Nerode's]  $A$  正则当且仅当  $A$  存在有限等价类划分, 其中  $x \sim_A y \Leftrightarrow (\forall z, xz \in A \Leftrightarrow yz \in A)$ .

[A4] CFL 对取  $\cap$  不封闭, 对取  $\cup$  封闭. **Eg.**  $\{0^n 1^n 2^*\} \cap \{0^* 1^n 2^n\}$ .

[A5] CFL 和正则的  $\cap$  仍然是 CFL. **Prf.** 把 DFA 揉到 PDA 里.

[A6] 前缀封闭 ( $xy \in A \Rightarrow x \in A$ ) 的无穷 CFL 存在无穷正则子集. **Prf.** 泵,  $uv^i$  ( $|v| > 0$ ) 或者  $uxy^iz$  ( $|v| = 0$ ) 正则.

[A7 22']  $\{0^a 1^b 2^a : 0 \leq a \leq b \leq 3a\}$  不是 CFL. **Prf.** 泵  $0^p 1^p 2^p$ , 讨论  $vxy$  位置.

[A8 Rice's] 对任意非平凡 (存在一个可识别语言满足, 也存在一个不满足) 的关于语言的命题  $P$ ,  $\{\alpha : P(L(M_\alpha))\}$  不可判定.

[A9] 不可识别:  $E_{TM}, EQ_{TM}, \overline{EQ_{TM}}, \overline{HALT}, UC, \overline{A_{TM}}$ .

[A10] 队列自动机 QA 等效于 TM. **Prf.** 互相模拟.

[A11] 给定单带  $M$ , 存在对  $M$  运行历史的编码方式, 设  $A = \{H : M \text{ accepted as history } H\}$  包含所有正确记录  $M$  运行到接受的历史的字符串 (至多一个元素), 则  $\overline{A}$  是 CFL. **Prf.**

$\mathcal{H}(C_0, \dots, C_k) = \#q_0 @ w_0 \# q_1 @ w_1^R \# \dots \#$ , 非确定性判断每种错误. **Eg.** 因此 CFG 全性不可判定, 否则  $A_{TM}$  可判定.

[A12] **NP** 对  $\cap$  和  $\cup$  都封闭. **Prf.** 把两个 NDTM 拼装.

[A13]  $f(n) = o(n \log n)$ , 则  $DTIME(f(n))$  只包含正则语言.

[A14]  $DP := \{L_1 \cap L_2 : L_1 \in NP, L_2 \in coNP\}$ , 则 EXACT-INDSET 是 **DP**-完全的.

[A15] SPACE-TMSAT (TM 在给定空间接受输入) 和 LINEAR-SPACE-TMSAT (TM 在线性空间接受输入) 都是 **PSPACE**-完全的.

[A16] 若允许 **NL** 验证机反复读证书, 则它强化为 **NP**. **Also:** 可以对数空间地检查单步格局转移正确性.

[A17 Immerman-Szelepcsényi's]  $\overline{L} \leq_1 \text{PATH} \Leftrightarrow L \leq_1 \overline{\text{PATH}}$ .

[A18] 所有  $f: \{0, 1\}^n \rightarrow \{0, 1\}$  都能用  $\mathcal{O}(2^n/n)$  规模电路计算. **Prf.** 朴素的  $\mathcal{O}(2^n)$  决策树的最低  $\log_2 n$  层电路门数超过了本质不同的函数数, 将它们合并.

[A19] 存在神谕  $A$  使得  $\mathbf{P}^A \neq \mathbf{NP}^A \wedge \mathbf{NP}^A \subset \mathbf{P}^A/\text{poly}$ .

[A20] 对任意  $k > 0$ ,  $\mathbf{PH}$  (更强地,  $\Sigma_4^P$ ) 中包含电路复杂性为  $\Omega(n^k)$  的语言.

[A21]  $\text{Add}_n: \{0, 1\}^{2n} \rightarrow \{0, 1\}^{n+1}$  和  $\text{Maj}_n: \{0, 1\}^n \rightarrow \{0, 1\}$  均能在  $\mathcal{O}(n)$  规模电路计算.

[A22]  $\mathbf{uNC}^1 \subset \mathbf{L}$ , 进而空间分层出  $\mathbf{uNC}^1 \subsetneq \mathbf{PSPACE}$ . **Prf.** DFS.

[A23]  $\text{Maj} \notin \mathbf{AC}^0$  (否则改装成 THR, counting, 直接枚举有多少个 1 就计算了 XOR);  
 $\text{Add}, \text{Cmp} \in \mathbf{AC}^0$ ,  $\text{Sum} \in \mathbf{AC}^1$ ,  $T_{w,\theta} \in \mathbf{NC}^1$  ( $w$  是常数, 直接复制输入若干份, pad 后用 Maj),  
 $\text{Maj} \in \mathbf{NC}^1$  (用全加器不断 3 to 2 归并), 进而  $\mathbf{AC}^0 \subsetneq \mathbf{TC}^0 \subseteq \mathbf{NC}^1$  (TC 是 AC with threshold).

[A24] 对  $p \in \mathbb{Q}$ ,  $\mathbf{BPP}_p = \mathbf{BPP}$ , 但对  $p \in \mathbb{R}$  存在反例.

[A25] 深度  $k$  的电路计算  $\text{XOR}_n$  的规模紧下界是  $2^{\Omega(n^{1/(k-1)})}$ .

[A26] Complexity Zoo

$$\begin{aligned} \mathbf{L} \subset \mathbf{NL} = \mathbf{coNL} \subset \mathbf{P} \subset \{\mathbf{BPP}, \mathbf{NP}, \mathbf{coNP}\} \subset \mathbf{PH} \\ \subset \mathbf{PSPACE} = \mathbf{NPSPACE} = \mathbf{IP} \subset \mathbf{EXP} \subset \mathbf{NEXP} = \mathbf{MIP}. \end{aligned}$$

- $\mathbf{NL} = \mathbf{coNL}$  (Immerman-Szelepcsényi's). **Prf.** 归纳计数.
- $\mathbf{NL} \subset \mathbf{uNC}^2$  ([B6]).
- $\Sigma_i^P = \Pi_i^P \Rightarrow \mathbf{PH} = \Sigma_i^P$ . **Eg.**  $\mathbf{NP} = \mathbf{coNP} \Rightarrow \mathbf{BPP} = \mathbf{NP}$ .
- $\mathbf{BPL} \subset \mathbf{nAC}^1$  ([B8]).
- $\mathbf{BPL} \subset \mathbf{L}^2 \cap \mathbf{P}$ .
- $\mathbf{RP} \subset \mathbf{NP}$ ,  $\mathbf{coRP} \subset \mathbf{coNP}$ ,  $\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP}$ .
- $\mathbf{NP} \subset \mathbf{P}/\text{poly} \Rightarrow \mathbf{PH} = \Sigma_2^P$  (Karp-Lipton's). **Prf.** 在这一假设下证明  $\Pi_2\text{SAT} \leq_p \Sigma_2\text{SAT}$ :  
取判定 SAT 的  $C$ , search-to-decision 地构造直接生成证书的  $C'$ ,  
 $\forall u \exists v \phi(u, v) \Leftrightarrow \exists C' \forall u \phi(u, C'(\phi, u))$ .
- $\mathbf{NP} = \mathbf{NuAC}^0$ ;  $\mathbf{NP}/\text{poly} = \mathbf{NnAC}^0$  ([B3]).
- $\mathbf{BPP} \subset \Sigma_2^P \cap \Pi_2^P$  ([B11]).
- $\mathbf{BPP} \subset \mathbf{P}/\text{poly}$  (Adleman's).
- $\mathbf{BPP} \subset \mathbf{ZPP}^{\text{SAT}}$ . **Prf.** 拆成  $\mathbf{RP}^{\text{SAT}}$  和  $\mathbf{coRP}^{\text{SAT}}$ , 用平移覆盖去随机.
- $\mathbf{NP} \subset \mathbf{BPP} \Rightarrow \mathbf{P}^{\text{SAT}} \subset \mathbf{BPP} = \mathbf{ZPP}^{\text{SAT}}$ . **Prf.** 将 SAT 神谕用  $\mathbf{BPP}$  机器实现, error reduction.
- $\mathbf{NP} \neq \mathbf{P} \Rightarrow \mathbf{P} \cup \mathbf{NPC} \subsetneq \mathbf{NP}$  (Ladner's).
- $\mathbf{BP} \cdot \mathbf{NP} \subset \mathbf{NP}/\text{poly}$ . **Prf.** error reduction, 取万能种子.
- $\mathbf{coNP} \subset \mathbf{NP}/\text{poly} \Rightarrow \mathbf{PH} = \Sigma_3^P$ . **Prf.** 证  $\Pi_3^P \subset \Sigma_3^P$ : 把  $\forall \exists \forall$  的最内层  $\forall$  用假设转换成  $\exists$ , 和前一合并, 最外层枚举电路结构得到  $\exists \forall \exists$ .
- $\mathbf{P} = \{\log\text{-space uniform circuits family}\}$ . **Prf.** 右含于左显然, 左含于右, 在格局图上 local 地算 (类似 Cook-Levin).

- $P = NP \Rightarrow EXP = NEXP$ . **Prf.** 给  $NEXP$  pad 为  $NP$ , 转换为  $P$ , 删去 pad 得到  $EXP$ .
- $NSPACE(S(n)) \subset DTIME(2^{O(S(n))})$ . **Prf.** 格局图上 BFS.
- $NSPACE(S(n)) \subset SPACE(S(n)^2)$  (**Savitch's**). **Prf.** 倍增搜可达性.
- $f(n) = o(g(n)) \Rightarrow SPACE(f(n)) \subsetneq SPACE(g(n))$  (**空间分层**).
- $f(n) \log f(n) = o(g(n)) \Rightarrow DTIME(f(n)) \subsetneq DTIME(g(n))$  (**时间分层**).
- $n < f(n) < f(n) \log^2 f(n) = o(g(n)) < g(n) = o(2^n/n) \Rightarrow SIZE(f(n)) \subsetneq SIZE(g(n))$  (**规模分层**). **Prf.** 取  $\ell = \log t_2 + \log \log t_2 + C$ , 存在需要至少  $2^\ell/\ell = \omega(f(n))$  规模电路才能计算的  $p: \{0, 1\}^\ell \rightarrow \{0, 1\}$ , 由于  $\ell < n$ , 可 pad 得到  $p': \{0, 1\}^n \rightarrow \{0, 1\}$ , 同时至多需要  $\ell 2^\ell = O(g(n))$  规模计算. (后者来自平凡上界, 所以应该可以用 [A18] 的技术做得更紧.)
- $MA \subset AM[2]$  ([B9]).
- $IP[k] \subset AM[k+2]$  (**Goldwasser-Sipser's**).
- 对任意常数  $k$ ,  $AM[2] = AM[k]$ . 考场上助教 恼羞成怒 禁止直接用这个结论, 建议补个完整证明.
- $IP_1 = IP$  (下标 1 表示完美完备性). **Prf.**  $IP = PSPACE$ , TQBF 由 sum-check 协议已经得到完美完备性.
- $MA_1 = MA$  ([B10]),  $AM_1 = AM$ . **Prf.** 后者还是平移覆盖展开为  $MAM$ , 然后  $MA \subset AM$  压缩回  $AM$ .
- $MA_0 = AM_0 = NP$ . **Prf.** 使 Arthur 接受的 Merlin 证据和随机种子可以直接作为证书.
- 任何要求完美可靠性的交互式证明至多描述  $NP$  (**Fürer-Goldreich-Mansour-Sipser-Zachos's** 补充).
- $PATH, \overline{PATH}$  都  $NL$ -完全. TQBF 是  $PSPACE$ -完全.

## 经典证明

[B1] 单带只读 TM 能力等同于 DFA.

**Prf.** 省流: 考虑一处交错序列, 它无法区分大量等价前缀的. 反证. 令  $Q_\circ := Q \sqcup \{\circ\}$ ,  $|Q_\circ| = n$ ,  $\#\{f: Q_\circ \rightarrow Q_\circ\} = n^n$  有限. 设  $m = n^{n+1} + 1$ , 由于  $A$  非正则, [A3] 给出存在  $m$  个等价类  $E = \{[x_1], \dots, [x_m]\}$  使得  $\bigcup_{i=1}^\infty [x_i] \subset A$ . 不妨  $|x_1| = \dots = |x_m| = \ell$ . 令  $\psi: E \rightarrow Q_\circ$  表示 TM 输入  $x \in E$  的初始格局开始, 当纸带指针第一次到达  $\ell + 1$  时, TM 的状态 (如果已经停机而不能到达  $\ell + 1$ , 则直接设为对应停机状态; 如果陷入死循环而不能到达  $\ell + 1$ , 则直接设为  $\circ$ ). 令  $\varphi: E \rightarrow Q_\circ \rightarrow Q_\circ$  表示 TM 从输入  $x \in E$ , 指针指向  $\ell$ , 状态为  $q \in Q_\circ$  的格局开始, 当指针第一次到达  $\ell + 1$  时, TM 的状态 (边界情况同上). 设  $m' = n^n + 1 = \lceil m/n \rceil$ . 由鸽巢原理, 存在  $E' = \{[y_1], \dots, [y_{m'}]\} \subset E$ , 使得  $\psi(y_1) = \dots = \psi(y_{m'})$ . 再由鸽巢原理, 存在  $\{u, v\} \subset E$ , 使得  $\varphi(u) = \varphi(v)$ . 而由于  $[u] \neq [v]$ , 存在  $z \in \Sigma^*$  使得  $uz$  和  $vz$  仅有一者被识别. 但是由于以上两个函数在  $u, v$  上相等,  $uz$  和  $vz$  在  $M$  上的每一步转移都相同, 应当  $M(uz) = M(vz)$ , 矛盾.

[B2] 若某个  $A \subset \{1\}^*$  是  $NP$ -完全的, 那么  $P = NP$ .

**Prf.** 设  $f: CNF \rightarrow \mathbb{N}$  满足  $\varphi \in SAT \Leftrightarrow 1^{f(\varphi)} \in A$ , DFS 求解 SAT 并用  $f$  做记忆化, 它能保证记忆化答案正确且总状态数是 poly.

[B3 22' 24']  $NP = NuAC^0$ ;  $NP/poly = NnAC^0$ .

**Prf.** 第一个,  $\supset$  显然.  $\subset$ , 给出 NDTM 的格局列表, 可以在  $NC^1$  或  $AC^0$  检查格局列表正确性. 非确定性枚举这个多项式长度列表. 第二个,  $\supset$  显然.  $\subset$ , 硬编码 advice, 然后同上.

[B4 22'] TQBF 在对数空间归约下 **PSPACE**-完全.

**Prf.**  $\text{TQBF} \in \text{PSPACE}$ : DFS. TQBF hard: 任意 **PSPACE** 的 TM 只有  $2^{\text{poly}(n)}$  个格局, 使用 Savitch 论证: 令  $\phi_i(C, C')$  表示  $C$  能否在  $2^i$  步转移到  $C'$ . 则

$$\phi_{i+1}(C, C') = \exists C'' \phi_i(C, C'') \wedge \phi_i(C'', C') = \exists C'' \forall (X, Y) \in \{(C, C''), (C'', C')\} \phi_i(X, Y).$$

显然可以对数空间计算这个函数.

[B5 22'] 若 PTM  $M_r$  以  $\varepsilon$  双边错误率判定  $L$ , 则存在  $2^{\mathcal{O}(\log \frac{n}{\delta})}$  大小的种子集  $S$  使得 ( $r \sim S$  即在集合中均匀采样):

$$\forall x \in \{0, 1\}^n \Pr_{r \sim S} [M_r(x) \neq L(x)] \leq \varepsilon + \delta.$$

**Prf.** 设  $M_r$  的随机串在样本空间  $\Omega$  中采样, 从  $\Omega$  中独立均匀抽  $t = \mathcal{O}((n+1)\delta^{-2})$  个串为  $S$ , 对固定的  $x$ , 设  $Z_i = [M_{r_i}(x) \neq L(x)]$ , 则  $\mathbb{E}[Z_i] < \varepsilon$ , Chernoff 给出  $Z_i$  均值大于  $\varepsilon + \delta$  的概率不超过  $e^{-\Omega(\delta^2 t)}$ , 可以取为  $2^{-n-1}$ . 最后对所有  $x$  union bound 得到, 取定  $S$  时, 存在  $x$  错误率超过  $\varepsilon + \delta$  的概率小于 1, 因而这样的  $S$  一定存在.

[B6 22']  $\text{NL} \subset \text{uNC}^2$ .

**Prf.** 只需证  $\text{PATH} \in \text{uNC}^2$ . 在 Bool 矩阵乘法  $(BC)_{ij} = \bigvee_k (B_{ik} \wedge C_{kj})$  下, 只需计算图的邻接矩阵幂  $A^{2^{\lceil \log N \rceil}}$ , 也即进行  $\log N$  次乘法, 每次乘法可以用  $\log$  层电路完成. 明所欲证.

[B7 22'] 给定  $S \subset \{0, 1\}^m$ , 其成员关系可以  $\text{poly}(m)$  地验证. 给定  $k \geq 2, 2^{k-2} < K \leq 2^{k-1}$ , 构造 **AM**[2] 协议使得以下两点都满足: (a)  $|S| \geq K$  时 Arthur 一定接受; (b)  $|S| \leq K/k^2$  时 Arthur 以至少  $2/3$  的概率拒绝.

**Prf.** 考虑仿射 hash 族  $\mathcal{H} := \{h : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^k\}$ , 其中  $h_{A,b}(x) = Ax + b, A \in \mathbf{M}_{k \times m}(\mathbb{F}_2), b \in \mathbb{F}_2^k$ . 令  $t = 4k$ , 构造协议: **(a)** Merlin 发  $t$  个  $h_1, \dots, h_t : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^k$ . **(b)** Arthur 均匀随机取  $y \in \mathbb{F}_2^k$  发回. **(c)** Merlin 回复下标  $i \in [t]$  和元素  $x \in \mathbb{F}_2^m$ . **(d)** Arthur 检查是否  $x \in S$  以及  $h_i(x) = y$ , 成功则接受, 否则拒绝.

**完备性:**  $|S| \geq K$ , 只需证明存在  $t = 4k$  个  $h$  使得  $\bigcup_{i=1}^t h_i(S) = \mathbb{F}_2^k$ . 令随机变量  $X_y = |\{x \in S : h(x) = y\}|$ , 则  $\mathbb{E}_{h \sim \mathcal{H}}[X_y] = \frac{|S|}{2^k} > \frac{1}{4}$ , 利用 **[C1]** 给出  $\Pr[X_y > 0] \geq \frac{\mathbb{E}[X_y]}{1 + \mathbb{E}[X_y]} \geq \frac{1}{5}$ , 即  $\Pr[y \notin h(S)] \leq \frac{4}{5}$ . 独立选  $t$  个再对所有  $y$  union bound 可知, 存在某个  $y$  未被覆盖的概率不超过  $2^k \left(\frac{4}{5}\right)^{4k} < 1$ , 因而满足条件的  $h$  族存在.

**可靠性:**  $|S| \leq K/k^2$ , 此时  $\left|\bigcup_{i=1}^t h_i(S)\right| \leq t|S|$ , 这样 Arthur 接受概率不超过  $\frac{4k \cdot K/k^2}{2^k} = \frac{4K}{k2^k} \leq \frac{2^{k+1}}{k2^k} = \frac{2}{k}$ . 不妨  $k \geq 6$  (常数的  $k$  可以直接让 Merlin 提供所有元素, Arthur 检查是否属于  $S$  且两两不同), 这时 Arthur 的接受概率就不超过  $\frac{1}{3}$ .

[B8 24']  $\text{BPL} \subset \text{nAC}^1$ .

**Prf.** 用 Adleman 定理的 trick (error reduction 配合完美种子存在性去随机化) 可知  $\text{BPL} \subset \text{nAC}^1 \Leftrightarrow \text{L} \subset \text{nAC}^1$ . 再用 **[B6]** 的 trick 给出  $\text{L} \subset \text{NL} \subset \text{nAC}^1$ .

[B9 24']  $\text{MA} \subset \text{AM}[2]$ .

**Prf.** 设原 Arthur 的证书长度为  $q(n)$ , 重复  $r = \mathcal{O}(q(n) + n)$  次取众数, error reduction 到  $2^{-q(n)-3}$  的双边错误率. 构造 **AM**[2] 协议: **(a)** Arthur 发送证书包  $Y$ ; **(b)** Merlin 返回  $z$ ; **(c)** Arthur 用 **MA** 的 Arthur  $V$  验证  $V(x, Y, z)$ .

**完备性:** 若  $x \in L$ , 发送 **MA** 对应的好证书  $z$ .

**可靠性:** 若  $x \notin L$ , 对抗性的 Merlin 给出的  $z$  总有

$$\Pr_Y[\exists z V(x, Y, z)] \leq \sum_{z \in \{0,1\}^{q(n)}} \Pr_Y[V(x, Y, z) = 1] \leq 2^{q(n)} \cdot 2^{-q(n)-2} < \frac{1}{3}.$$

或者直接  $\mathbf{MA} \subset \mathbf{AM}[4] = \mathbf{AM}[2]$ .

[B10 24']  $\mathbf{MA} = \mathbf{MA}_1$ , 其中  $\mathbf{MA}_1$  要求完备性中接受概率为 1.

**Prf.** 只需证  $\mathbf{MA} \subset \mathbf{MA}_1$ . 用 [B11] 的技术将  $\exists z \Pr \geq \frac{2}{3}$  拆成  $\exists z \exists (y_1, \dots, y_m) \forall Y \bigvee \dots = 1$ , 让 Merlin 提供前两个  $\exists$  的结果.

[B11 Sipser-Gács-Lautemann's]  $\mathbf{BPP} \in \Sigma_2^P \cap \Pi_2^P$  (平移覆盖去随机化).

**Prf.** 对  $L \in \mathbf{BPP}$ , error reduction 后得到  $M_r$  使用  $m = \text{poly}(n)$  的随机 bit 以  $\delta < \frac{1}{m+1}$  的双边错误率判定  $L$ . 记  $A_x = \{r : M_r(x) = 1\}$ . 断言 (平移覆盖技术):

$$\Pr_r[M_r(x) = 1] \geq 1 - \delta \Leftrightarrow \exists (y_1, \dots, y_{m+1}) \in \{0,1\}^m, \forall z \in \{0,1\}^m, \bigvee_{i=1}^{m+1} M_{y_i \oplus z}(x) = 1.$$

一方面, 若  $x \in L$ , 则  $|A_x| \geq (1 - \delta)2^m$ ,  $|\overline{A_x}| \leq \delta 2^m$ . 对于任意  $z$  和均匀独立采样的  $y$ ,  $\Pr[z \notin \bigcup_{i=1}^{m+1} (A_x \oplus y_i)] \leq \delta^{m+1}$ , union bound 就有  $\Pr[\exists z \notin \bigcup_{i=1}^{m+1} (A_x \oplus y_i)] \leq 2^m \delta^{m+1} < 1$ , 所以这时右侧为真. 另一方面, 若  $x \notin L$ , 直接 counting 就保证必然存在  $z$  反例. 到此  $\mathbf{BPP} \subset \Sigma_2^P$ . 又因为  $\mathbf{BPP}$  对补封闭, 就得到目标.

[B12] 集合下界协议板子 &  $\mathbf{GNI} \in \mathbf{AM}[2]$ .

集合下界协议: 设  $S_x = \{z \in \{0,1\}^N : \exists w R(x, z, w) = 1\}$ , 其中  $R$  是 Arthur 的多项式验证器, 并且假设有 promise  $|S_x| \geq L \vee |S_x| \leq L/4$ . 协议如下: **(a)** 取常数  $R_0 = 160$ , 令  $m = \lfloor \log_2(L/R_0) \rfloor$ , Arthur 随机选择仿射 hash 函数  $h_{A,b}(z) = Az + b : \{0,1\}^N \rightarrow \{0,1\}^m$ , 并发送给 Merlin. **(b)** Merlin 需要回复  $120 = 3R_0/4$  个两两不同的  $z_1, \dots, z_{120} \in S_x$  及其证书  $w_1, \dots, w_{120}$  满足  $h(z_i) = 0^m \wedge R(x, z_i, w_i) = 1$ . Arthur 检查这一点.

**Prf.** 令  $X = |\{z \in S_x : h(z) = 0^m\}|$ , 根据两两独立性,  $\mathbb{E}[X] = |S_x|/2^m$ ,  $\text{Var}[X] \leq \mathbb{E}[X]$  (因为是两两独立的 Bernoulli 实验). **完备性:** 若  $|S_x| \geq L$ , 则  $\mathbb{E}[X] \geq R_0 = 160$ ,  $\Pr[X < 120] \leq \text{Var}[X]/(160 - 120)^2 \leq 16/160 = 0.1$ , Merlin 有至少 0.9 的概率成功. **可靠性:** 若  $|S_x| \leq L/4$ , 则  $\mathbb{E}[X] < 80$ , 这时  $\Pr[X \geq 120] \leq \text{Var}[X]/(120 - 80)^2 \leq 8/160 = 0.05$ , 因此 Merlin 只有不超过 0.05 的概率可以作弊成功.

**Eg.** 对  $\mathbf{GNI}$ , 令  $\text{Orb}(G) = \{\pi(G) : \pi \in \mathfrak{S}_n\}$ ,  $\text{Aut}(H) = \{\alpha \in \mathfrak{S}_n : \alpha(H) = H\}$ ,

$$\mathcal{S} = \{(H, \alpha) : H \in \text{Orb}(G_0) \cup \text{Orb}(G_1), \alpha \in \text{Aut}(H)\}.$$

轨道-稳定子定理给出  $G_0 \simeq G_1 \Rightarrow |\mathcal{S}| = n!$ ,  $G_0 \not\simeq G_1 \Rightarrow |\mathcal{S}| = 2n!$ . 取  $\mathcal{T} = \mathcal{S}^2$ , 这样就有 4 倍差距的 promise, 直接套板子.

[B13] Sum-check 协议.

对于 SAT: 将 CNF 改写成  $P(x_1, \dots, x_n) \in \{0, 1\}$  的多项式, 欲判定  $\sum_{x_{1:n}} P(x)$  是否为  $c$ . 先要求 Merlin 给一个素数  $p \in [2^n, 2^{2n}]$  (用 Millar-Rabin 检查), 此后在  $\mathbb{F}_p$  下计算. 每次向 Merlin 询问  $s(x_1) := \sum_{x_{2:n}} P(x_{2:n}; x_1)$  这个多项式, 检查是否  $s(0) + s(1) = c$ , 然后采样  $a \in \mathbb{F}_p$  并递归检查是否  $s(a; x_{2:n}) = \sum_{x_{2:n}} P(a; x_{2:n})$ . 单次正确率至少  $1 - \frac{\mathcal{O}(n)}{p}$ .

对于 TQBF, 需要引入  $\forall$  对应的  $\prod$  和降低度数的算子  $L_i(P) = x_i P|_{x_i=1} + (1 - x_i) P|_{x_i=0}$ , 且需要处理乘积上溢出  $p$  导致的误判 ( $\exists$  替换为  $P(0) + P(1) - P(0)P(1)$ ). 最终也能得到具有完美完备性的协议.

**[B14] 交叉序列技术:**  $f(n) = o(n \log n)$  的  $\text{DTIME}(f(n))$  只含正则语言.

**Prf.** 反证. 设单带 TM  $M \in \text{DTIME}(f(n))$ ,  $f(n) = o(n \log n)$  而  $M$  识别的  $A$  不正则. 记  $\text{cs}_x(i)$  表示将  $x$  作为输入时,  $M$  的读写头穿过纸带  $i$  和  $i + 1$  边界时的机器状态按照运行时间升序排列得到的状态序列, 即交叉序列,  $|\text{cs}_x(i)|$  表示其长度.

先证引理: 若  $x = uvw$  满足  $|u| = p$ ,  $|uv| = q$ , 且  $\text{cs}_x(p) = \text{cs}_x(q)$ , 则  $M(x) = M(uw)$ . 这是因为,  $|\text{cs}_x(p)| = |\text{cs}_x(q)|$  给出  $M$  在停机 (或者陷入死循环) 时一定不处于  $v$  内部, 那么  $M$  在  $v$  内部的运行行为完全由离开  $v$  的状态决定, 即由  $v$  两端的交叉序列. 而  $v$  两端交叉序列与  $\varepsilon$  相同, 所以  $M(x) = M(uvw) = M(u\varepsilon w) = M(uw)$ .

由于  $A$  不正则, 根据 Myhill-Nerode 定理,  $A$  中存在无穷多个前缀等价类. 因此, 对任意大的  $n \in \mathbb{N}$ , 总存在一个  $x$ , 它是其所属等价类中最短的代表元且满足  $|x| \geq n$ . 根据引理, 若  $x$  内部存在两个位置产生了相同的交叉序列, 则这两个位置之间的串可以删去得到  $x'$ , 且  $x' \equiv x$ , 与  $x$  的最短性矛盾. 所以  $x$  内部的交叉序列两两不同. 据此, 考虑  $M(x)$  的运行时间

$$T(n) \geq \sum_{i=1}^n |\text{cs}_x(i)| \geq \min_{\{c_j\}} \left\{ \sum_{j=0}^{\infty} j \cdot c_j \mid \sum_{j=0}^{\infty} c_j = n \wedge (\forall j, c_j \leq |Q|^j) \right\}.$$

不妨设  $|Q| = s$ ,  $n = s^{k+1} - 1$ , 则  $T(n) \geq \sum_{j=0}^k j s^j \geq k s^k = \Theta(kn) = \Theta(n \log n)$ , 矛盾.

**[B15] 神谕世界的对角化技术:** 存在神谕  $A$  使得  $\mathbf{P}^A \neq \mathbf{NP}^A \wedge \mathbf{NP}^A \subset \mathbf{P}^A/\text{poly}$ .

**Prf.** 取  $A = S \oplus T := \{0x : x \in S\} \sqcup \{1y : y \in T\}$ , 其中  $T = \text{TQBF}$  满足  $\mathbf{P}^T = \mathbf{NP}^T$ . 对于  $S$ , 构造如下: 列出所有多项式时间的 OTM  $M_1, M_2, \dots$ , 设它们的时间上界为多项式  $p_1, p_2, \dots$ , 令  $S_0 = \emptyset$ , 此后归纳地构造. 对  $k \in \mathbb{Z}_{\geq 1}$ , 取足够大的  $n_k$  满足  $n_k > \max\{n_i : 1 \leq i \leq k-1\}$  且  $2^{n_k} > p_k(n_k)$ . 讨论  $M_k^{S_{k-1} \oplus T}(0^{n_k})$  的运行结果:

- 若  $M_k$  接受, 则  $S_k := S_{k-1}, D_k := D_{k-1}$ .
- 否则, 由于  $p_k(n_k) < 2^{n_k}$ , 一定存在  $x_k^* \in \{0, 1\}^{n_k}$  没有被  $M_k$  在运行过程中询问过. 取其中字典序最小者, 令  $S_k := S_{k-1} \cup \{x_k^*\}$ .

最后取  $S := \bigcup_{k \geq 1} S_k$ . 以下说明这样构造的  $A$  满足题设. 对于  $\mathbf{P}^A \neq \mathbf{NP}^A$ , 考虑对角线构造:

$L := \{0^n : \exists x \in \{0, 1\}^n, 0x \in A\}$ . 一方面,  $L \in \mathbf{NP}^A$ , 这是因为我们可以用 NDTM 非确定性枚举所有长为  $n$  的串, 然后调用预言机验证结果. 另一方面, 如果存在多项式的 OTM  $M = M_k$  判定  $L$ , 则考虑  $M_k^A(0^{n_k})$  的结果:

- 若  $M_k$  拒绝了  $0^{n_k}$ , 根据定义, 长为  $n_k$  的  $x_k^*$  已然被加入  $A$  中, 应当有  $0^{n_k} \in L$ , 判定错误.
- 若  $M_k$  接受了  $0^{n_k}$ , 由于  $x_k^*$  是  $S$  中唯一可能长为  $n_k$  的串, 必然就有  $x_k^* \in S_k \subset S$ . 但这意味着  $M_k^{S_{k-1} \oplus T}$  其实拒绝了  $0^{n_k}$ , 且由于运行时间限制,  $M_k^{S_{k-1} \oplus T}$  的行为应当和  $M_k^A$  一致, 这与  $M_k$  接受  $0^{n_k}$  矛盾.

总之, 不存在多项式的 OTM 判定  $L$ ,  $\mathbf{P}^A \neq \mathbf{NP}^A$ .



对于  $\mathbf{NP}^A \subset \mathbf{P}^A/\text{poly}$ , 考虑任意 ONDTM  $N^A$ , 设其时间界为多项式  $p$ . 由于  $S$  是稀疏的, 当给定输入长度  $n$ , 设  $\ell = p(n)$ ,  $N$  至多向预言机询问长度不超过  $\ell$  的串, 因此, 令建议  $a(n)$  硬编码集合  $\{x : x \in S \wedge |x| \leq \ell\}$ , 集合字符串总长不超过  $\ell^2$ , 因而编码能在多项式内完成. 每当  $N$  向预言机询问  $0x$  时,  $N/a(n)$  直接遍历  $a(n)$  以检查  $x$  是否被编码, 即可用这个建议代替语言的功能. 这样,  $N^A = N^{S \oplus T} = N^T/a(n)$ . 而根据  $T$  的定义,  $\mathbf{P}^T = \mathbf{NP}^T$ , 则存在 OTM  $M^T$  能够模拟  $N^T$ . 那么  $M^T/a(n)$  模拟了  $N^A$ . 最终就有  $\mathbf{NP} \subset \mathbf{P}^A/\text{poly}$ .

## 数学工具

---

**[C1]**  $X \geq 0, \mathbb{E}[X^2] < \infty, \mathbb{E}[X] > 0$ , 则  $\Pr[X > 0] \geq \frac{\mathbb{E}[X]^2}{\mathbb{E}[X^2]}$  (可用于 **AM** 协议构造时的概率分析).

**[C2 Chernoff's]** 若  $X_i \in [0, 1]$  且  $X_1, \dots, X_n$  互相独立, 那么  $\Pr\left[\frac{1}{n} \sum X_i \geq \mu + \delta\right] \leq e^{-2\delta^2 n}$  ( $\delta \geq 0$ ).